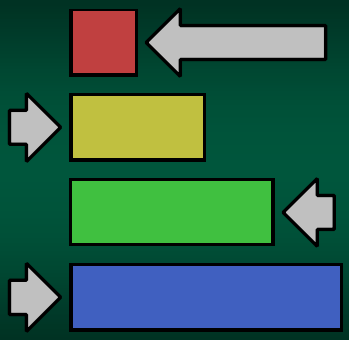
Recursive Sorting

Part 9

1



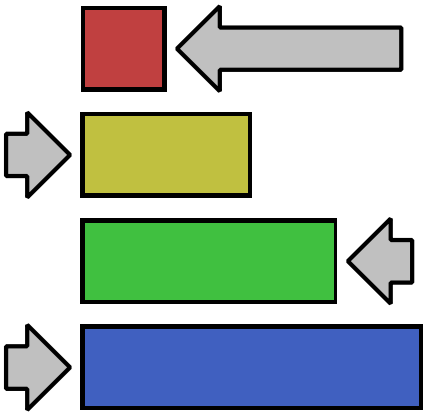
Merging Arrays

Quite easy... and quite common

2

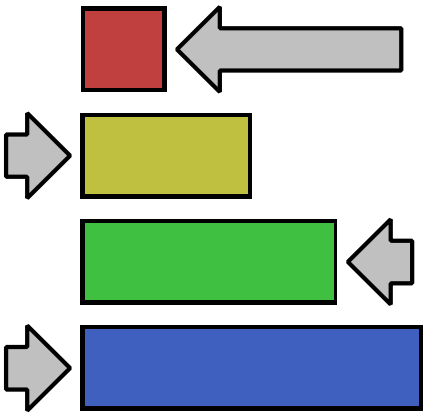
Merging Arrays

- It is a common task in Computer Science to combine two different arrays into one
- If both arrays are unsorted…
  - the task is fairly simple O(n)
  - just add one onto the end of the other



Merging Arrays

- However, often two sorted arrays are combined
- ...and the resulting array must be sorted



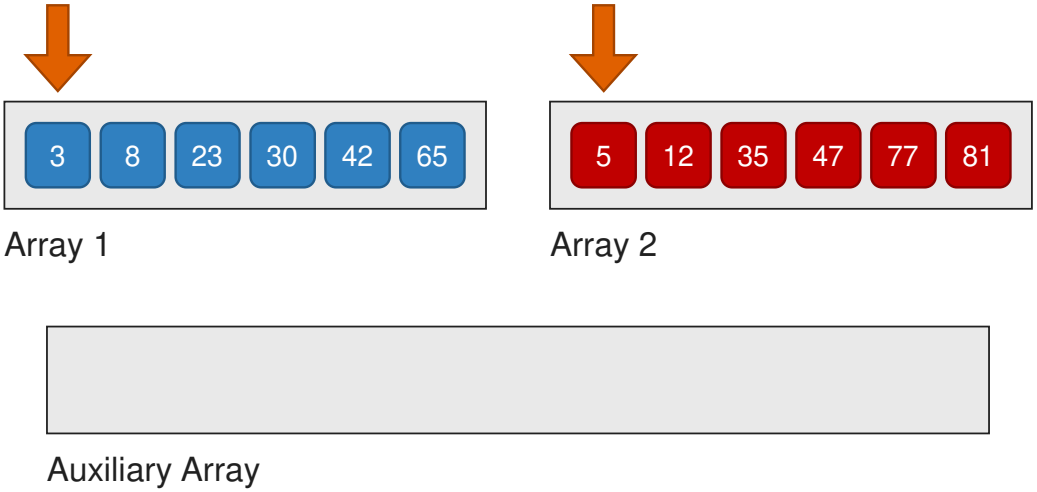
Merging Arrays

- The algorithm for merging two sorted arrays is very simple
- The resulting time complexity is O(n)
- However, it requires auxiliary storage of O(n)

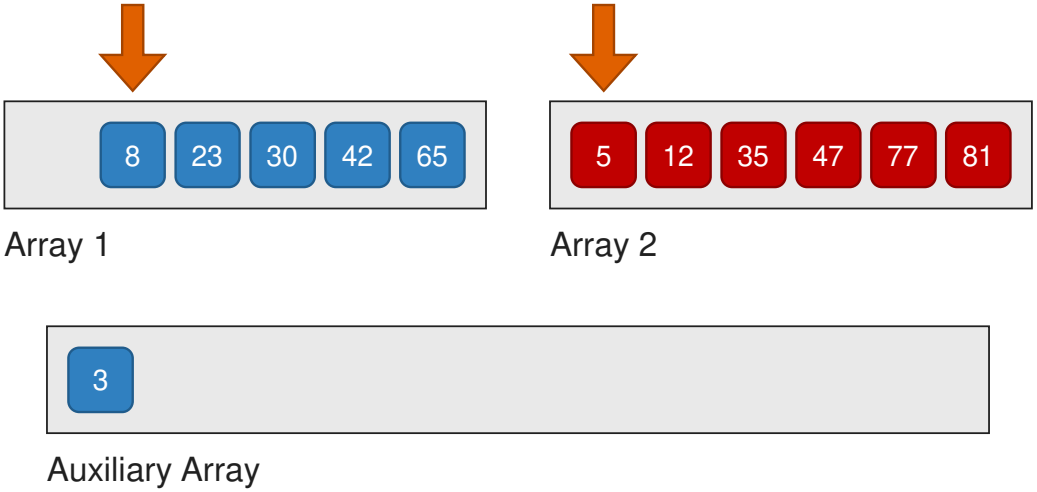
Merge Algorithm

- Keep two counters – one for each array
- Loop while both arrays have data
  - take the smaller element and put it in the auxiliary array
  - increment the array's counter (which just lost an element)
- After the loop
  - one array will still have elements
  - append them to the auxiliary array

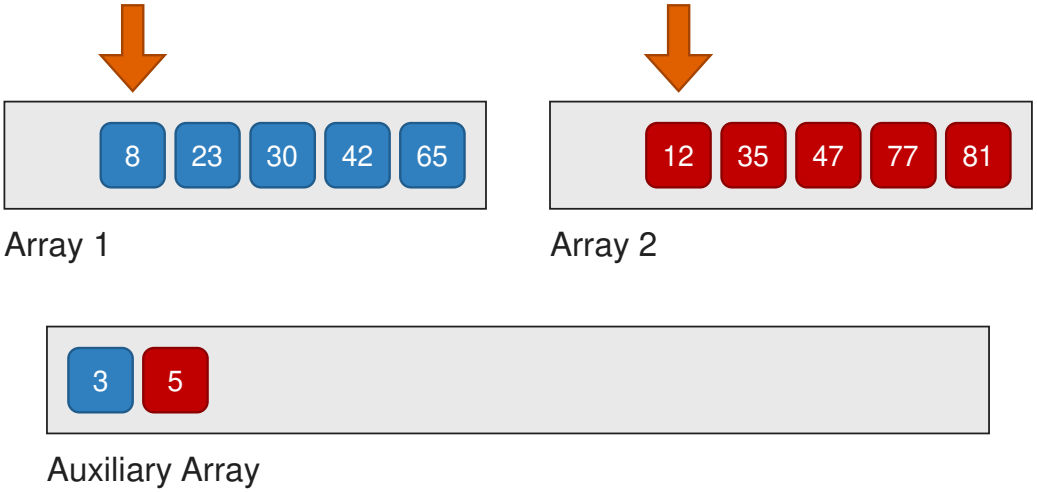
Sorted Array Merge Example



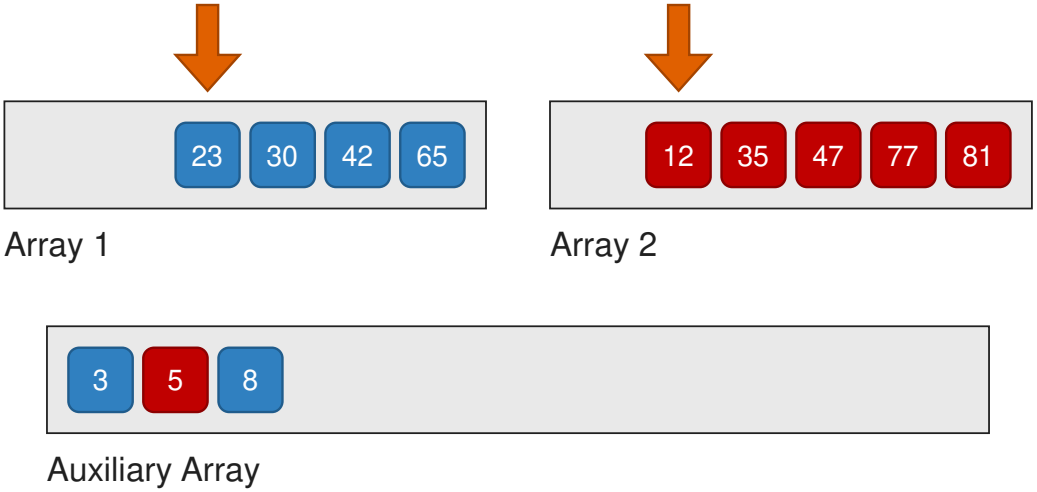
Sorted Array Merge Example



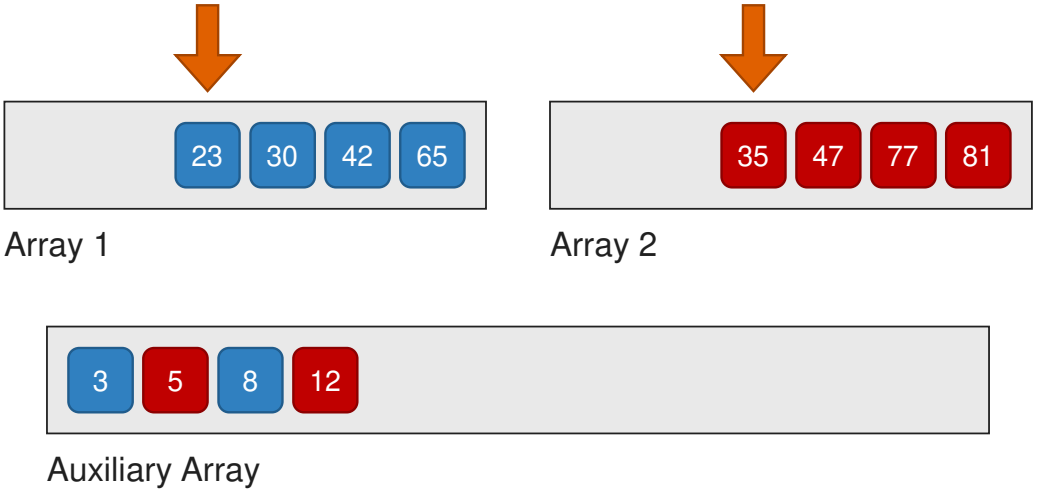
Sorted Array Merge Example



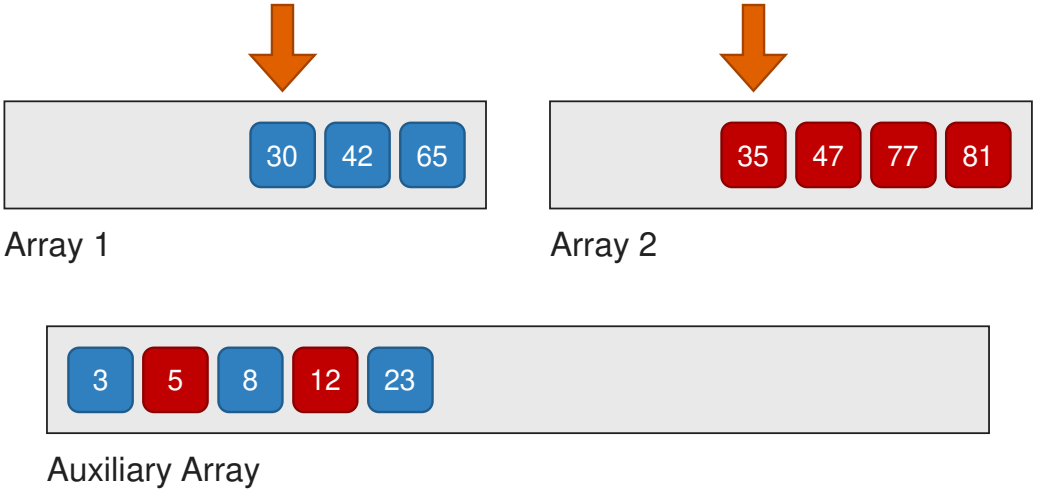
Sorted Array Merge Example



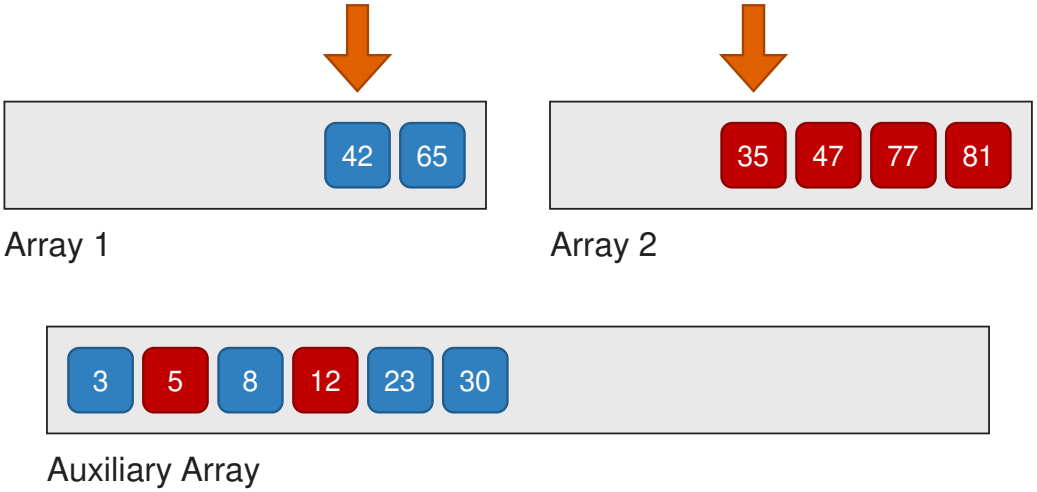
Sorted Array Merge Example



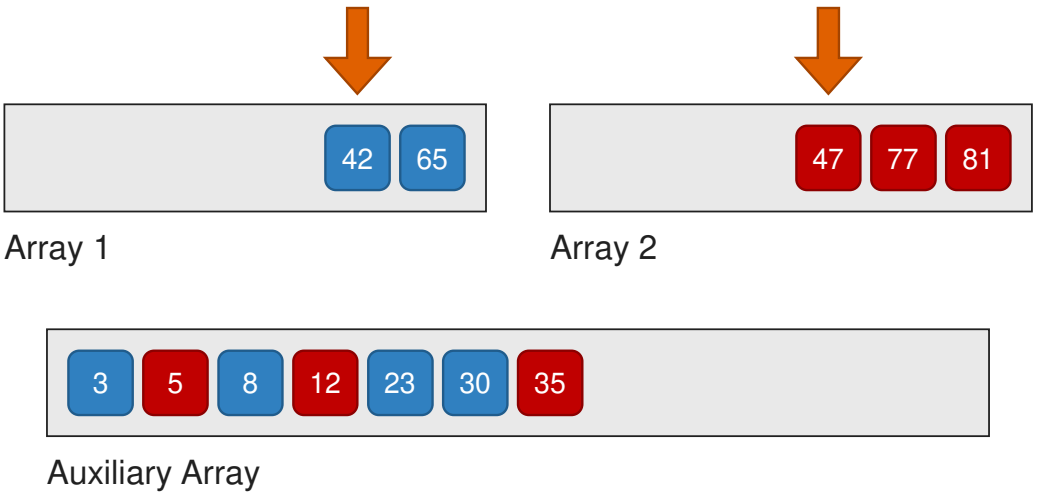
Sorted Array Merge Example



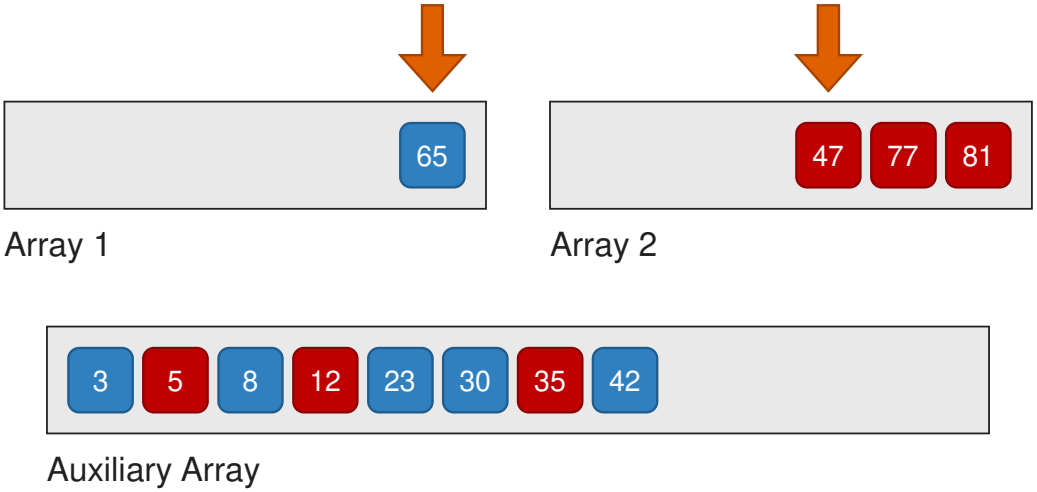
Sorted Array Merge Example



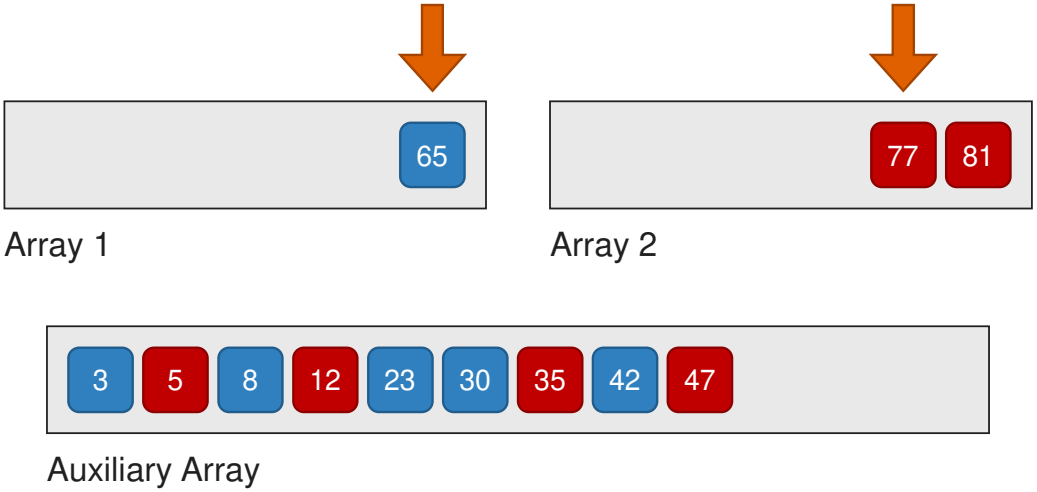
Sorted Array Merge Example



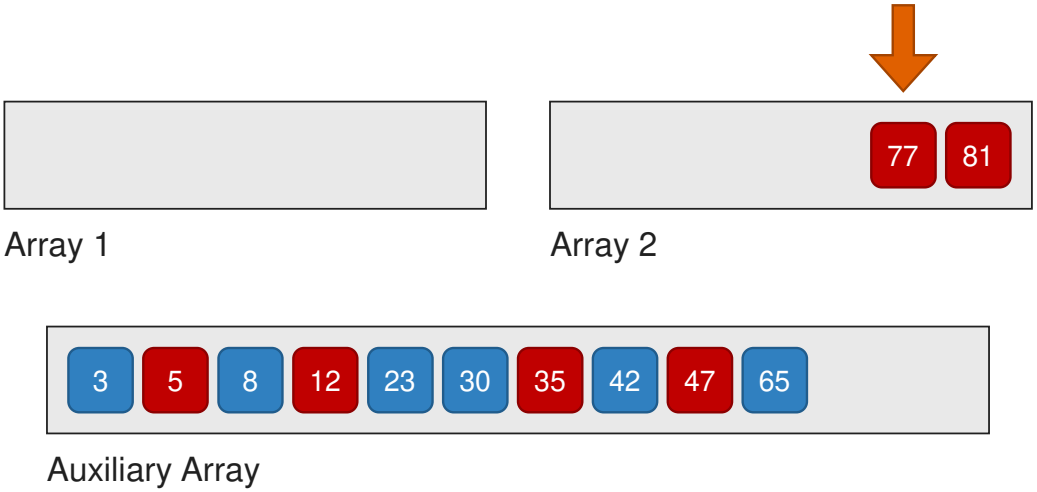
Sorted Array Merge Example



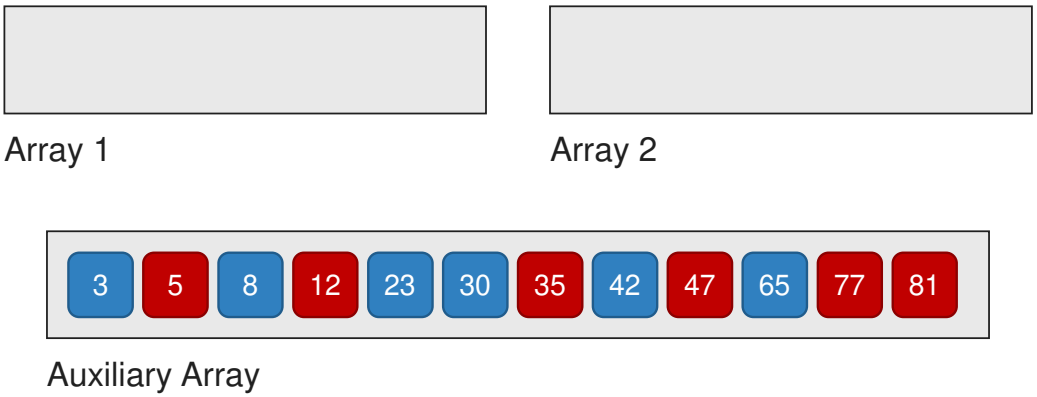
Sorted Array Merge Example

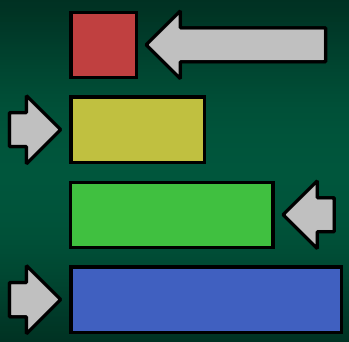


Dump Rest of the Array



Dump Rest of the Array





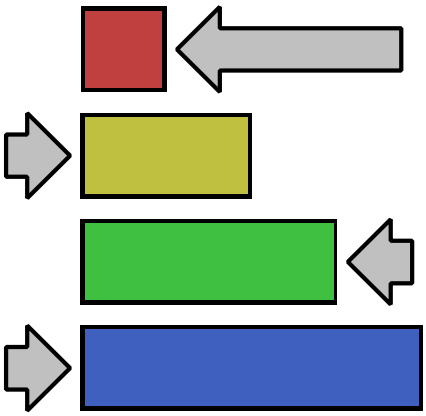
Merge Sort

Divide and conquer!

19

Merge Sort

- *Merge Sort* is a divide-and-conquer algorithm that cuts an array into smaller and smaller sublists until sorting them is arbitrary
- Invented by *John von Neumann* in 1945 *(19 BBW)*



20

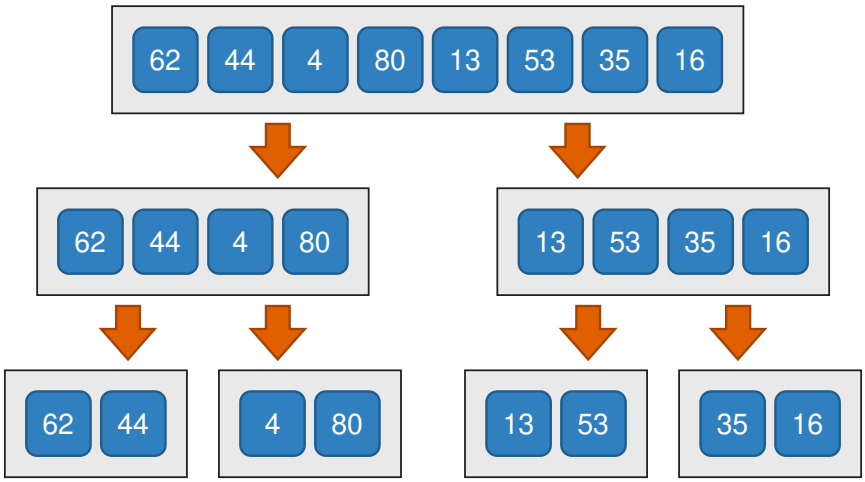
Merge Sort

- Because Merge-Sort divides the list into smaller instances of itself, it naturally is solved using recursion
- Each recursive step cuts the list into two sublists until…
  - the list has 2 elements – arbitrary swap
  - the list has 1 element – which is, well, sorted

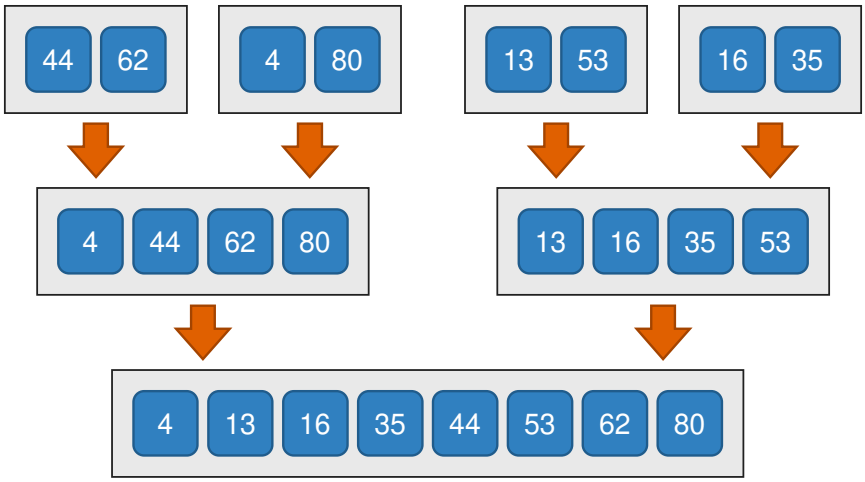
Merge Sort

- As the recursion bubbles up, each sub list is merged using the algorithm we just discussed
- Divide-and-conquer algorithms ultimately result in O(n log n)
- Since an auxiliary array is required for the merge process, Merge-Sort, while fast, has O(n) auxiliary storage requirements

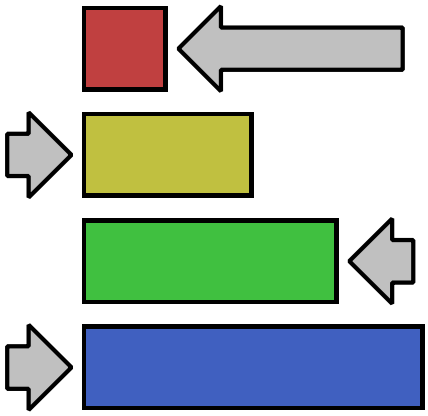
Merge Sort Example: Recurse down



Sort Merge Sort Example: Merge Up

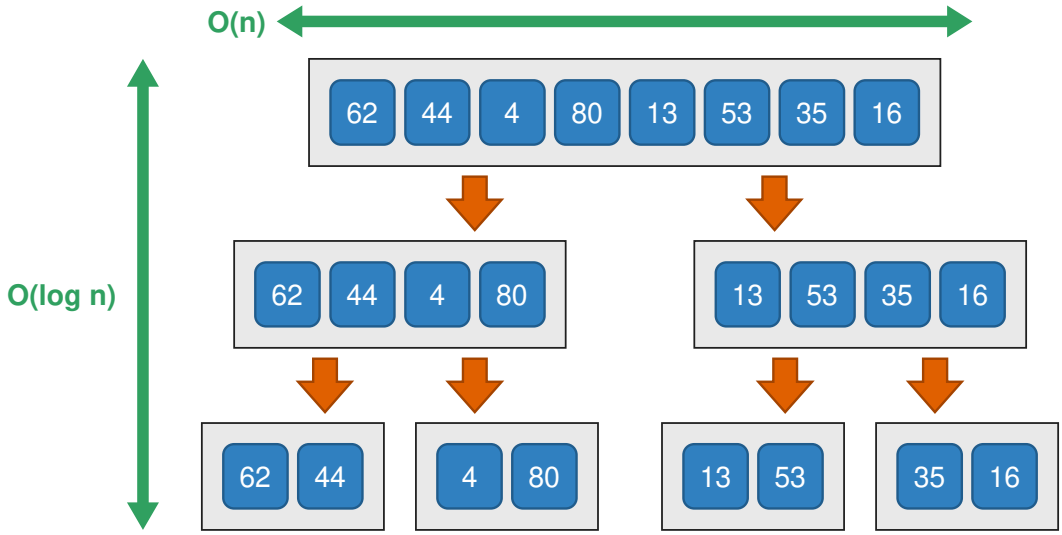


Merge Sort



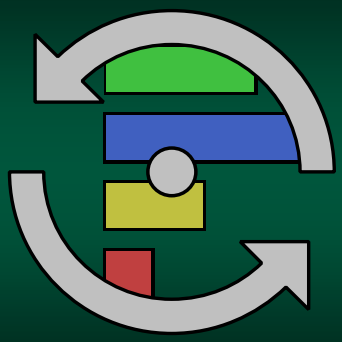
- Merge Sort is O(n log n)
- The number of recursive calls is equal to $\log_2$ the number of items: O(log n)
- At each level of recursion, all items are analyzed: O(n)

Merge Sort is O(n log n)



Merge Sort Summary

Merge Sort	
Time Average	O(n log n)
Time Best	O(n log n)
Time Worst	O(n log n)
Auxiliary space	**O(n)**
Stable	Yes – Equal element order preserved
Online?	Yes – New data → new sublist

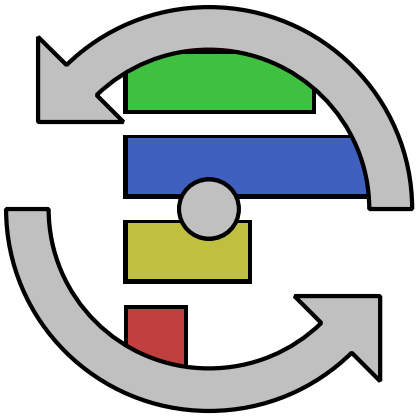


Quick Sort

Oh, I am getting dizzy....

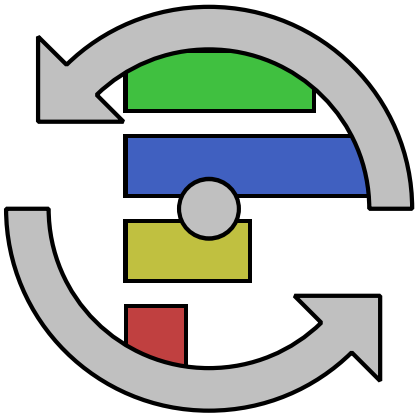
Quick Sort

- *Quick-Sort* is a divide-and-conquer algorithm that rotates values around a *pivot*
- Invented by *C. A. R. Hoare* in 1959 *(5 BBW)*
- Even faster than both Merge Sort and Heap Sort
- ... but has a weaknesses



How it Works

- Like Merge-Sort, the array is broken down into smaller and smaller sub-lists
- However, before recursion
  - a value *p* is chosen in the sub-list as the *pivot* value
  - smaller items are moved before it
  - larger items are moved after it



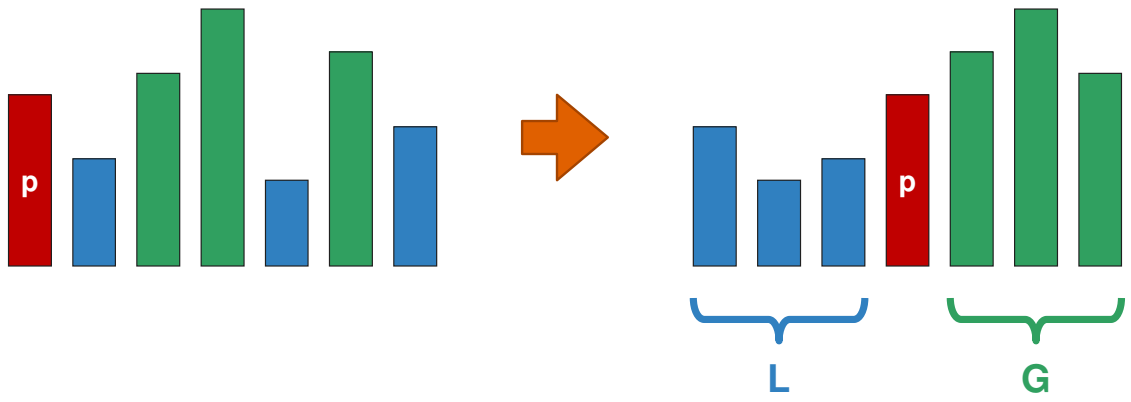
Choosing a Pivot

- Pivot can be <u>any</u> element in the sub-array
- …we need one <u>actual</u> value to compare
- This *pivot* is used to *partition* the values
- Different versions use different pivots
  - first item in the sub-array
  - end item in the sub-array
  - the midpoint of the sub-array
  - random value in the sub-array

Partitioning the Values

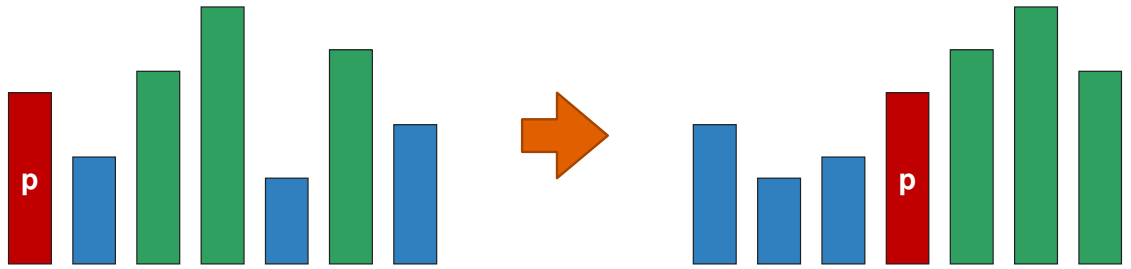
- After the pivot *p* is selected, all elements are moved
- Two, separate, loops move through the elements and swaps elements less than/greater than the pivot
- The result is...
  - sub-array **L** contains items less than *p*
  - sub-array **G** contains items greater than *p*

Partitioning (pivot is the first item)



Partitioning the Values

- Note: neither **L** or **G** is sorted yet
- These will be called recursively by Quick-Sort
- Moving the elements, in-place, can look a tad ugly code-wise, but the logic is straight forward



Partition Algorithm

- The sub-lists are stored in the original array – so there's no auxiliary storage
- The algorithm maintains two pointers
  - first moves left to right and keeps track of the values that are **too big**
  - second moves right to left and keeps track of the values that are **too small**
- Each moves independently

Partition Algorithm

- First move the **Too Big** pointer until a value is found that is bigger than the pivot
- Then move the **Too Small** pointer until a value is found that is smaller than Pivot
- Then, these values are swapped
- When the two pointers collide, we are done

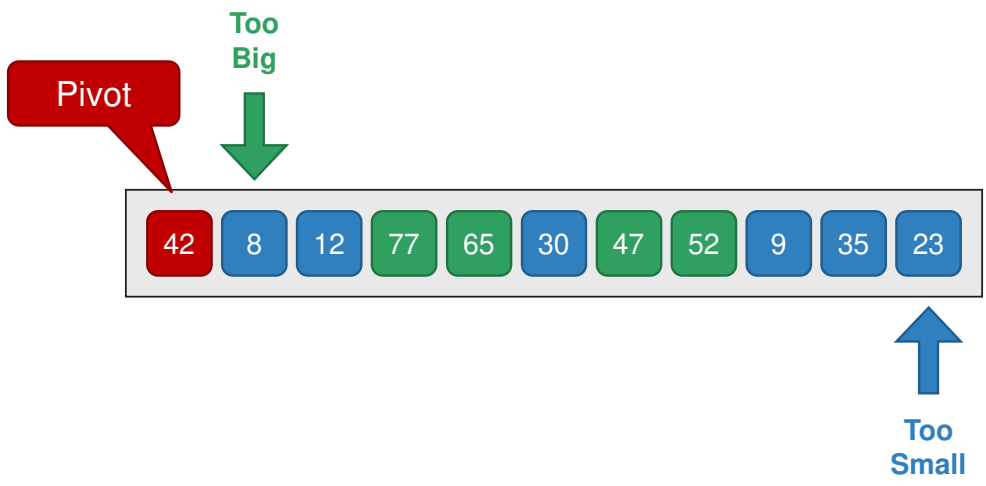
Example Partition

- *In this example*, we pivot at the start of the array
- Any value can be used...
  - but it will have to be swapped to the start before the algorithm runs
  - this "saves" the pivot for later

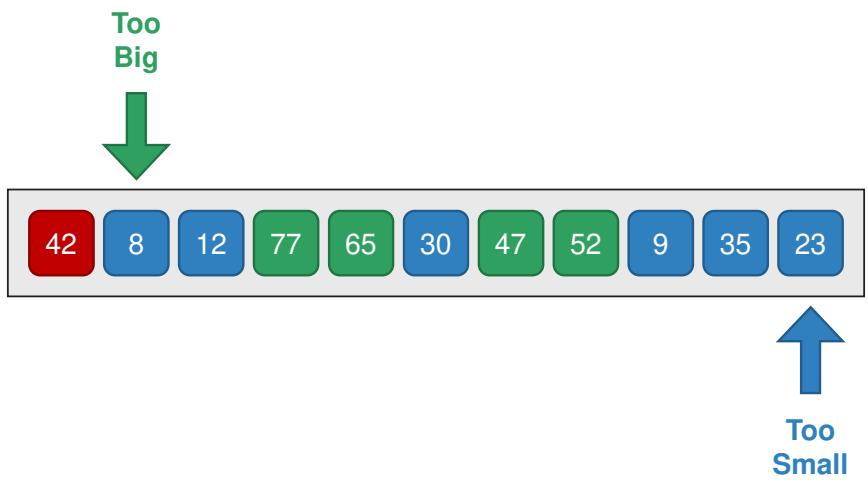
42	8	12	77	65	30	47	52	9	35	23


```
while tooBig < tooSmall
    while  array[tooBig] <= array[pivot]
        tooBig++
    end while

    while array[tooSmall] > array[pivot]
        tooSmall--
    end while

    if tooBig < tooSmall
        swap array[tooBig] and array[tooSmall]
    end if
end while

swap array[tooSmall] and array[pivot]
```

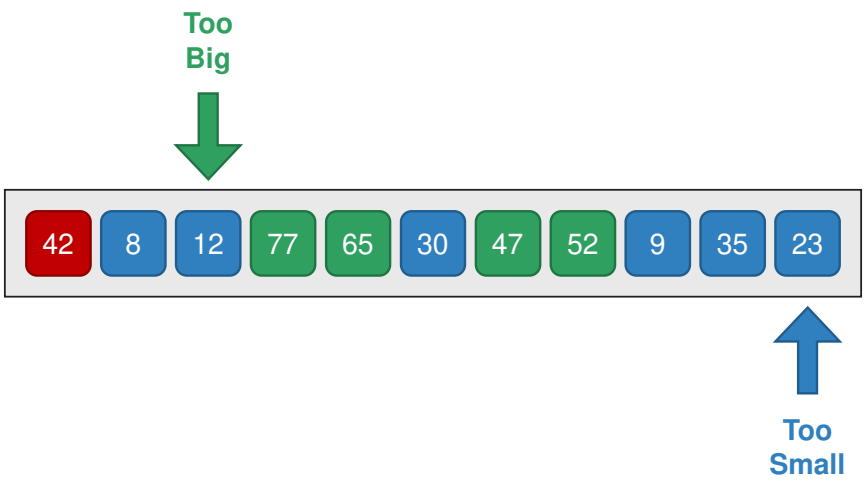
Example: Pivot is First



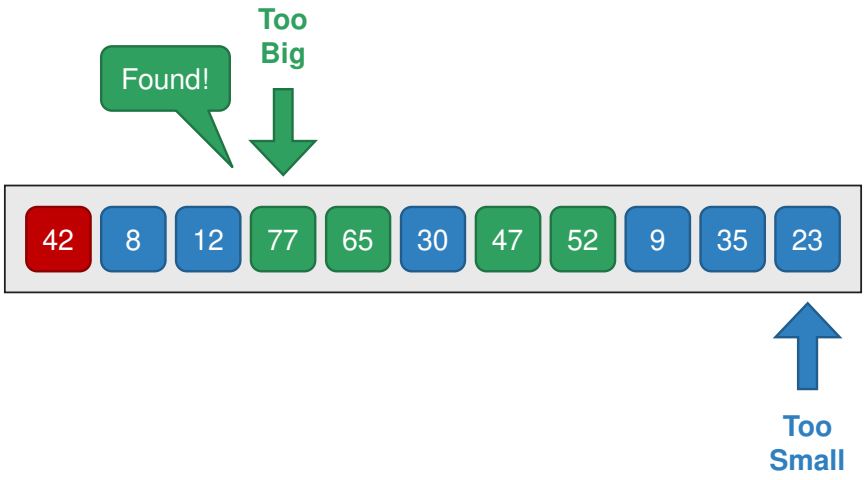
Example: Move Too Big



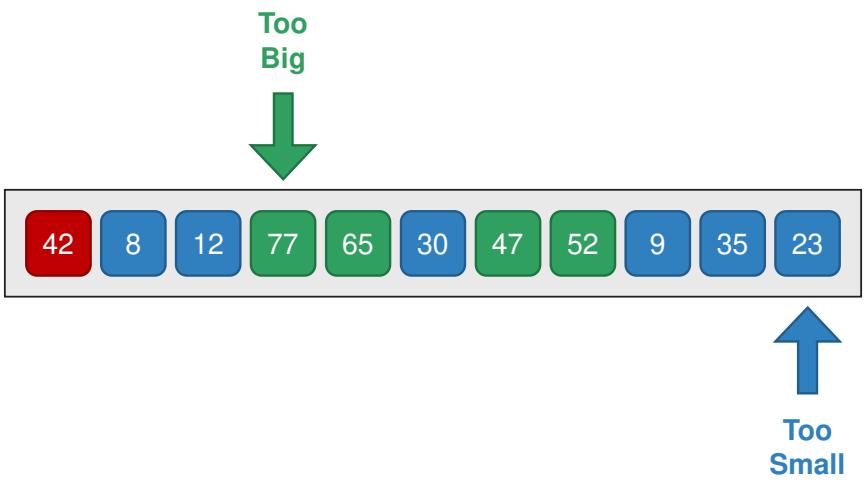
Example: Move Too Big



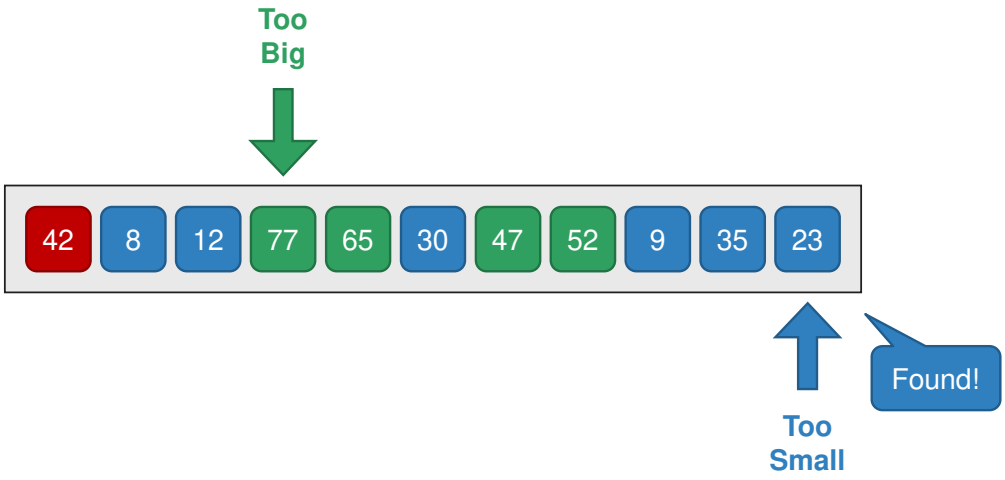
Example: Move Too Big



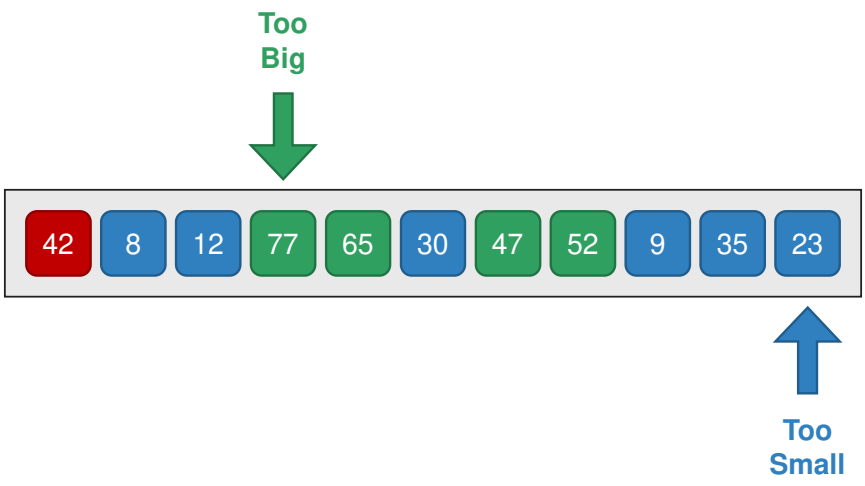
Example: Now, Move Too Small



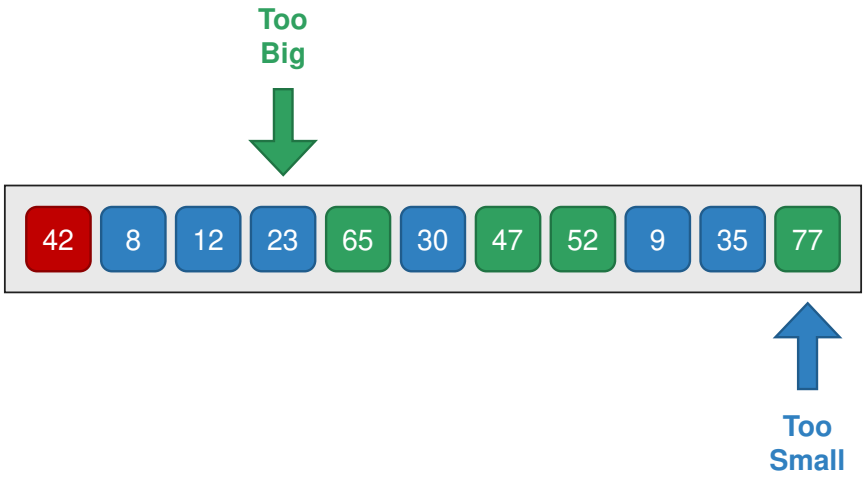
Example: Found (Immediately)



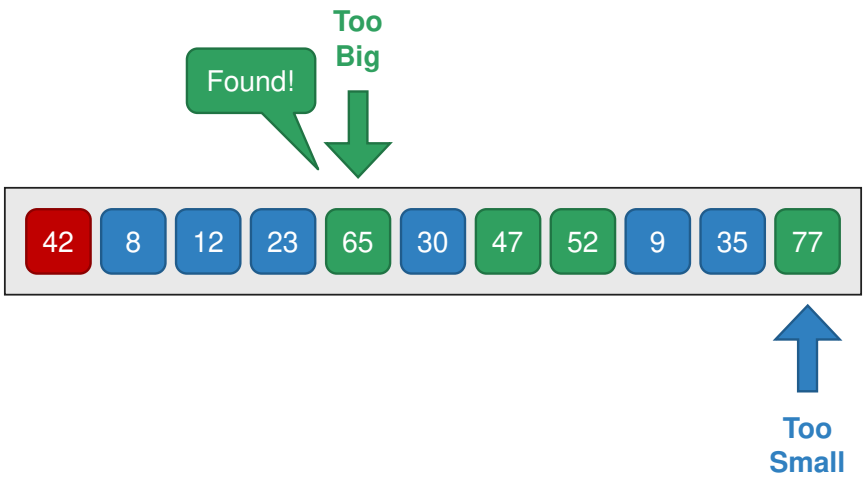
Example: Swap Values



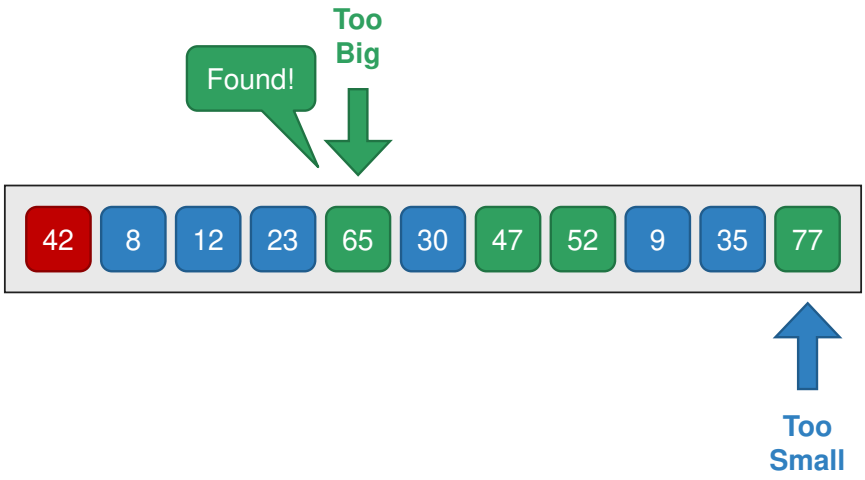
Example: Keep going… Move Too Big



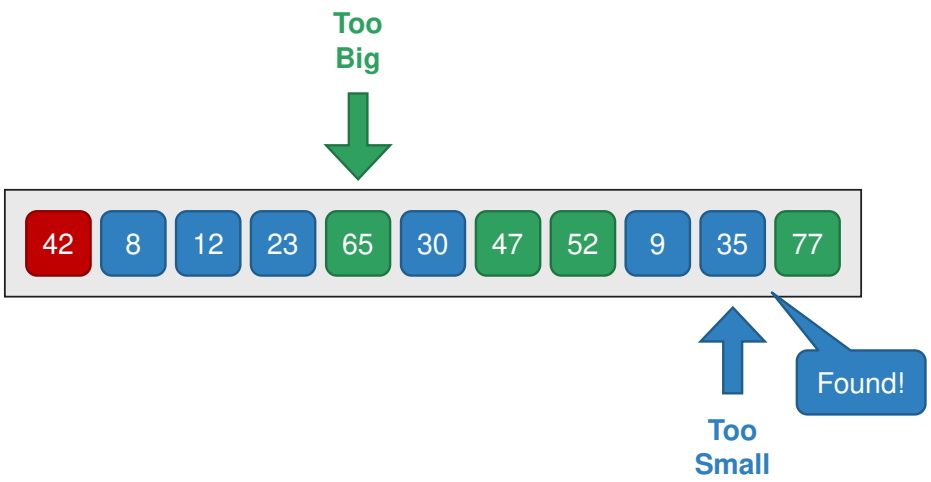
Example: Too Big Found



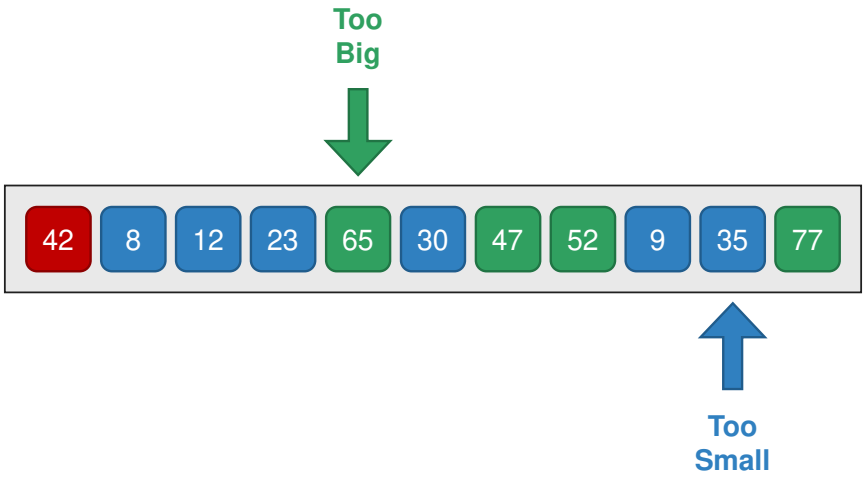
Example: Move Too Small



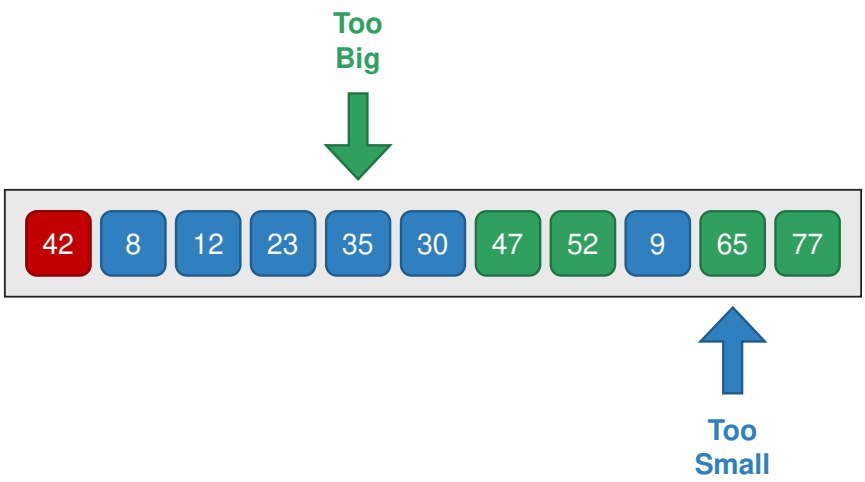
Example: Too Small Found



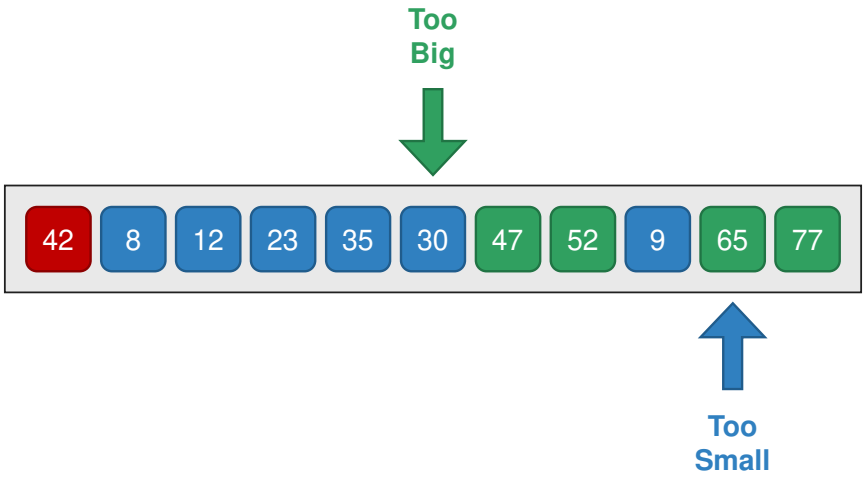
Example: Swap Values



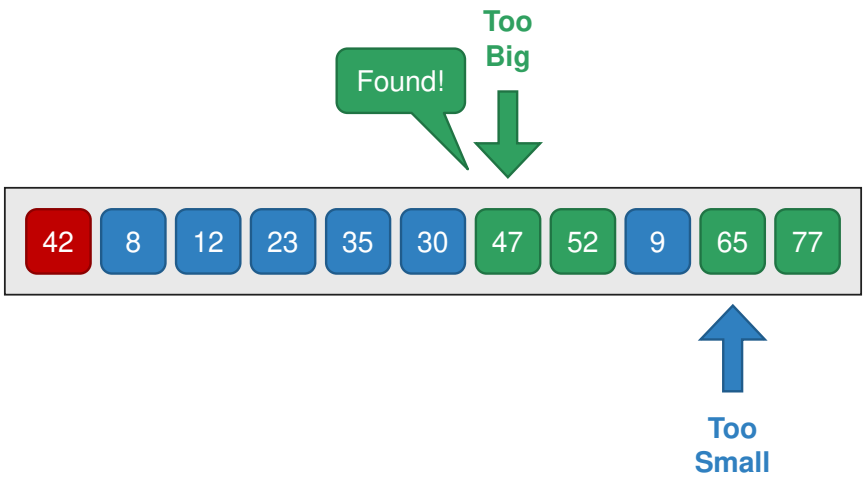
Example: Keep going… Move Too Big



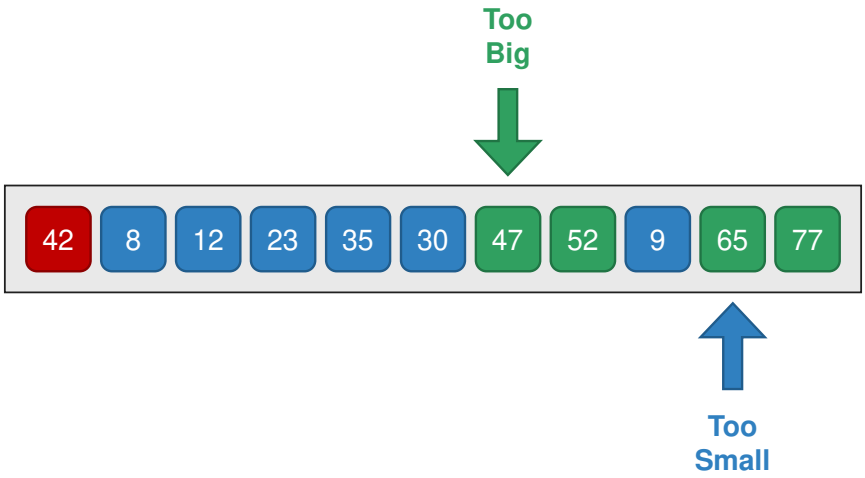
Example: Keep going… Move Too Big



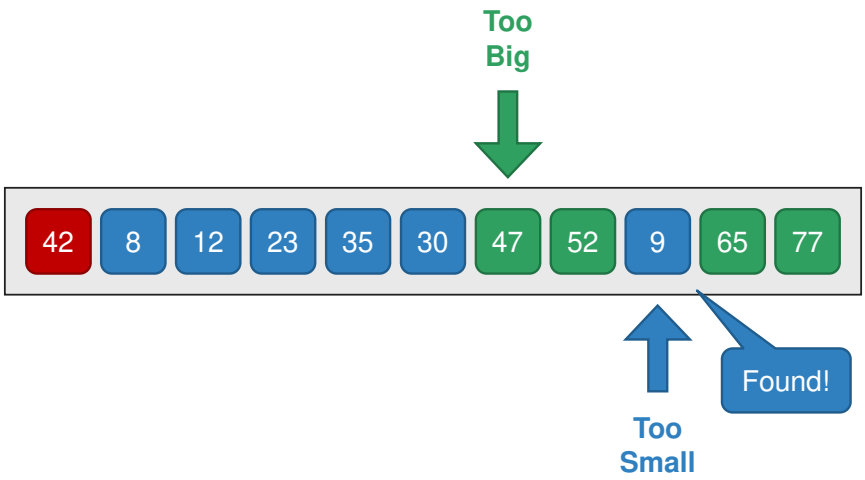
Example: Too Big Found



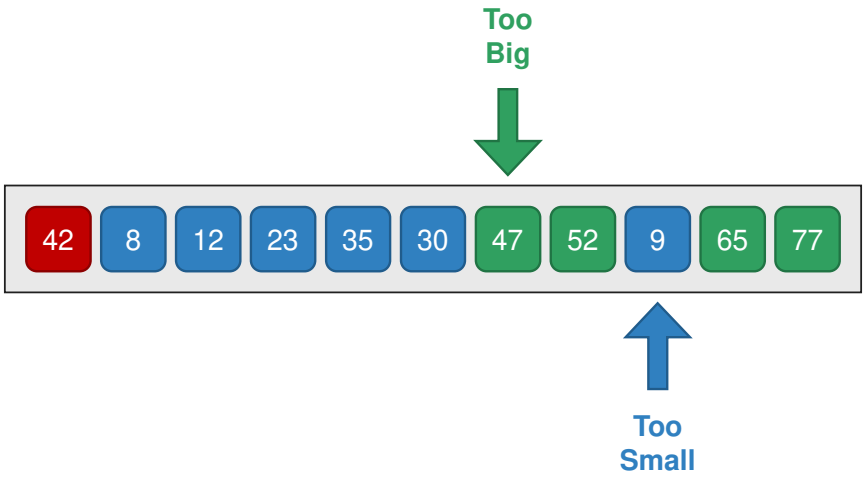
Example: Move Too Small



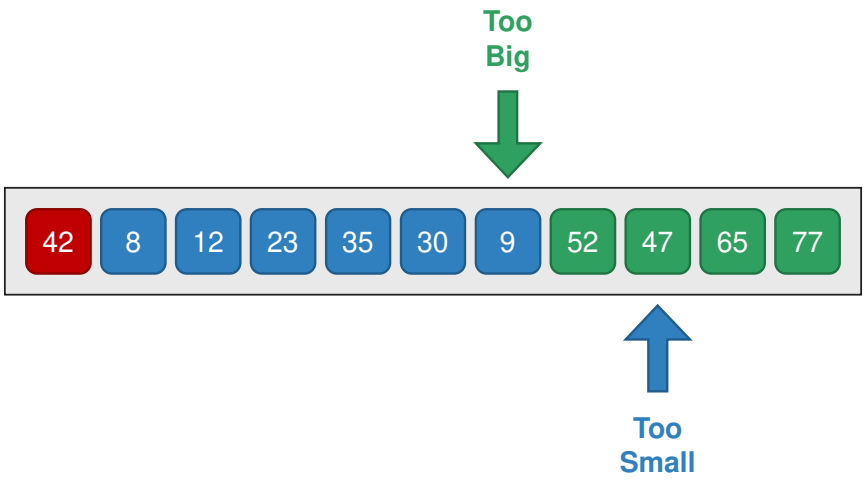
Example: Too Small Found



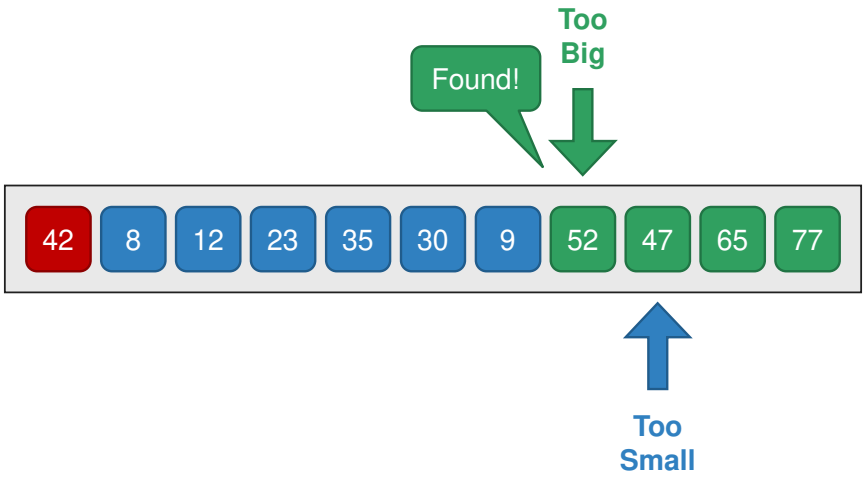
Example: Swap Values



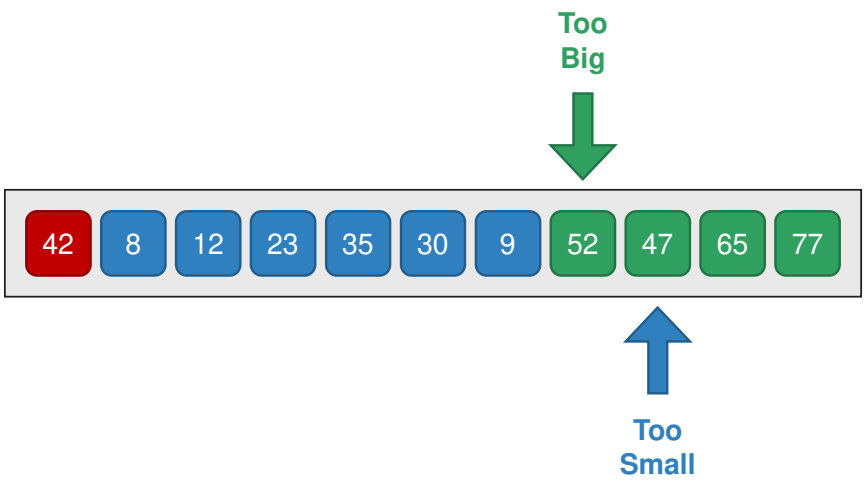
Example: Keep going… Move Too Big



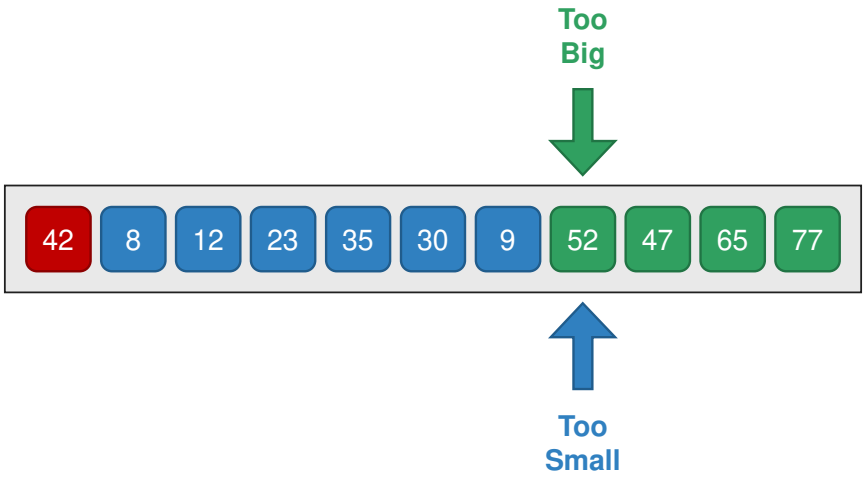
Example: Too Big Found



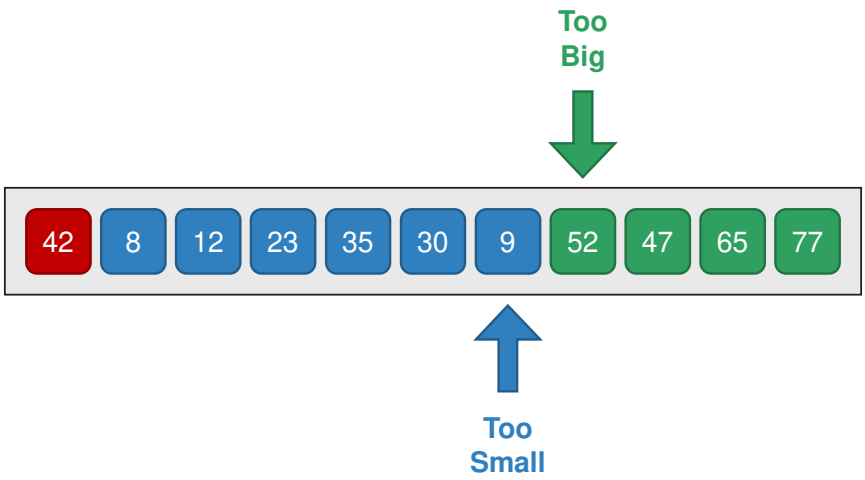
Example: Move Too Small



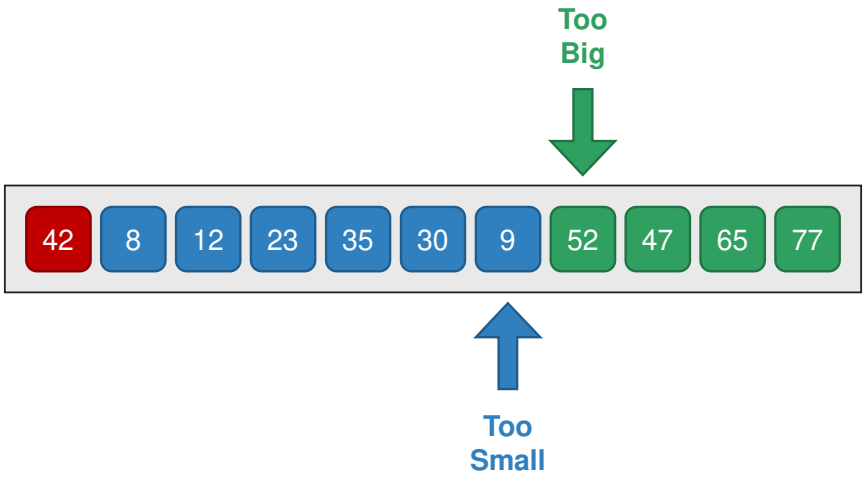
Example: Move Too Small



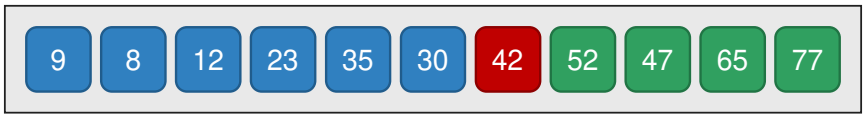
Example: Pointers Passed Each Other



Example: Swap Pivot & Too Small



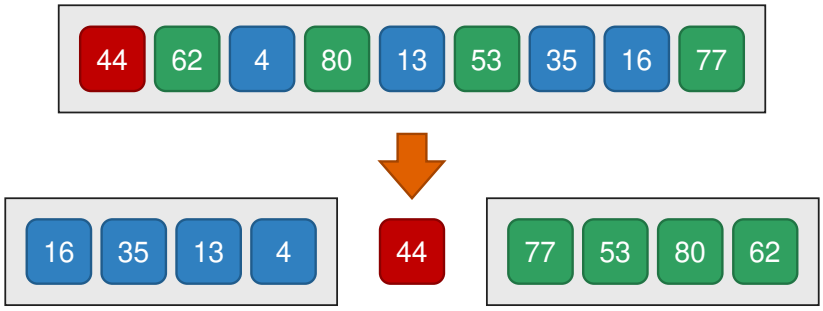
Example: Done (with this pass)



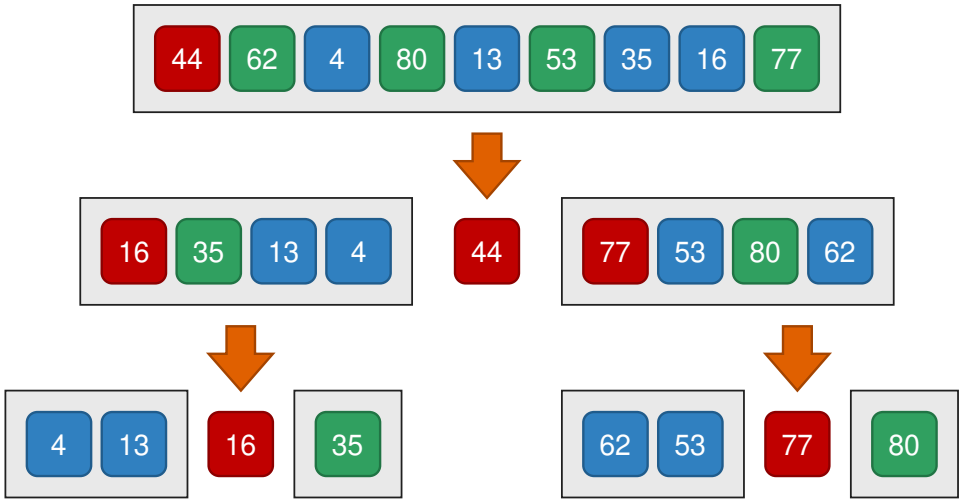
Recursion Time!

- Notice: all the items before the pivot are smaller and all the items after are a larger
- Now, we can recurse both sides
- The result is a sorted array

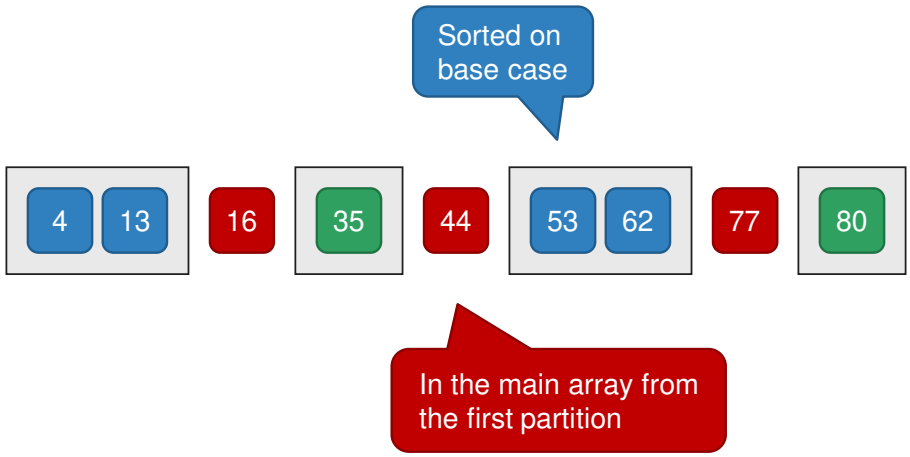
Quick Sort Example



Quick Sort Example



Quick Sort Example



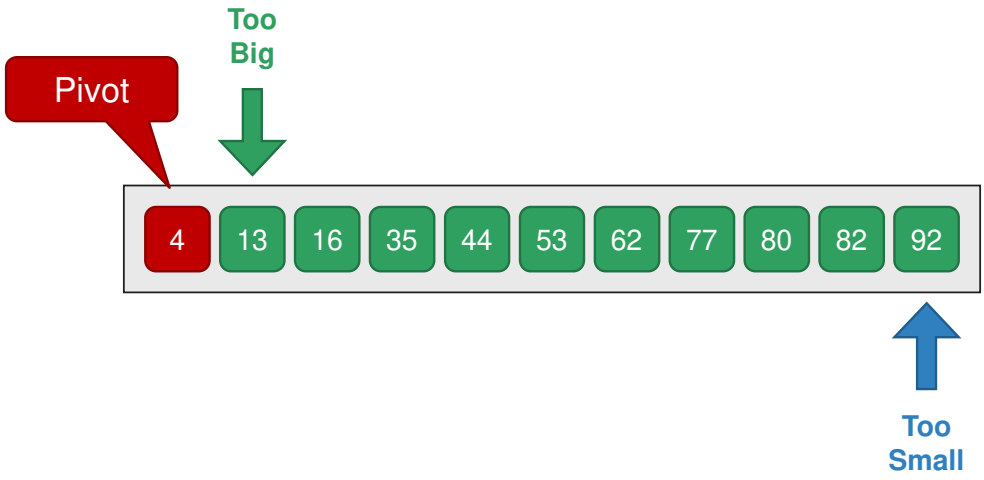
Quick Sort: Worst Case

- Assume we get array that is already sorted
- This can cause huge problems!
- Shockingly, the efficiently of this sort can degenerate if we are not careful

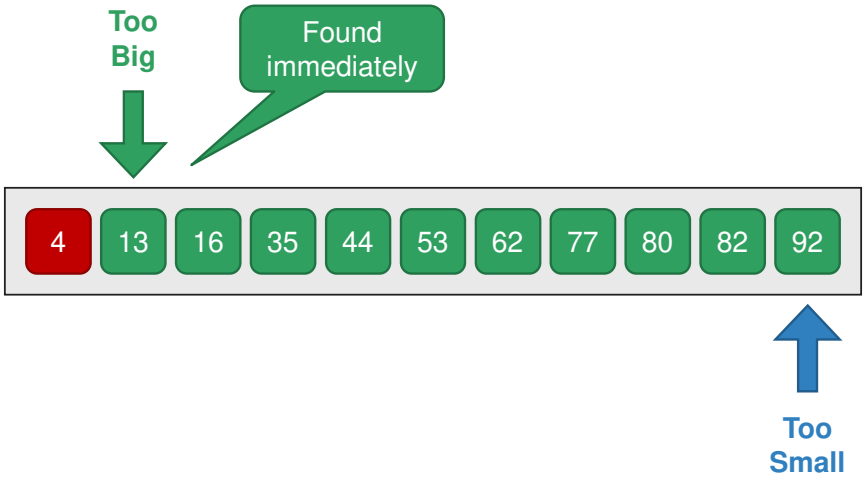
Quick Sort: Worst Case

- If the first item is the pivot
  - a sorted array will cause both the pointers will pass simply pass each other
  - one sub-array will be empty, the second will contains **ALL** the elements – 1
- If the last item is the pivot
  - reverse sorted array will have the same effect

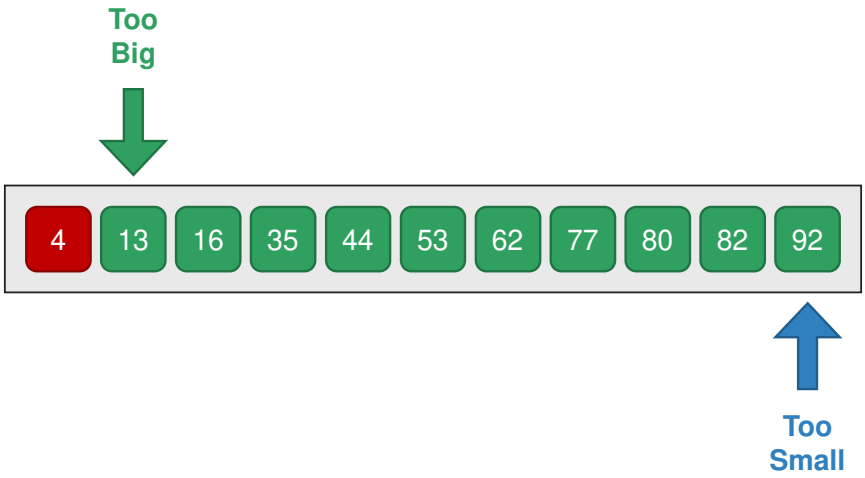
Quick Sort: Worst Case



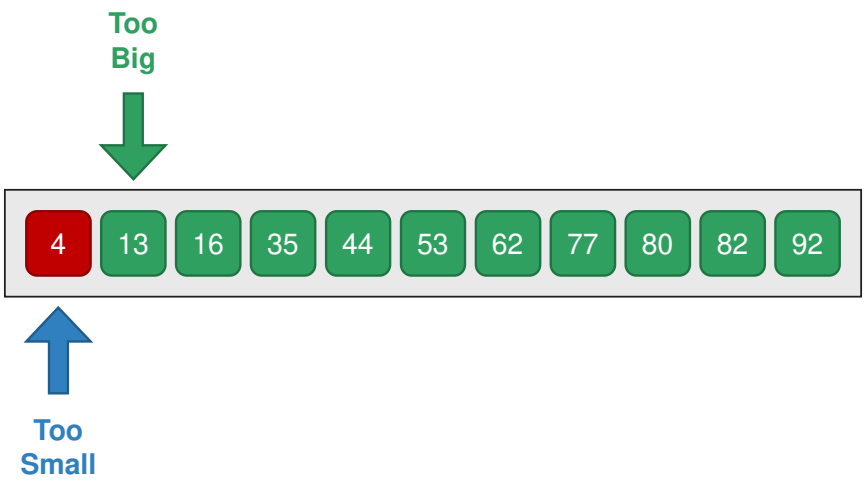
Worst Case: Move Too Big



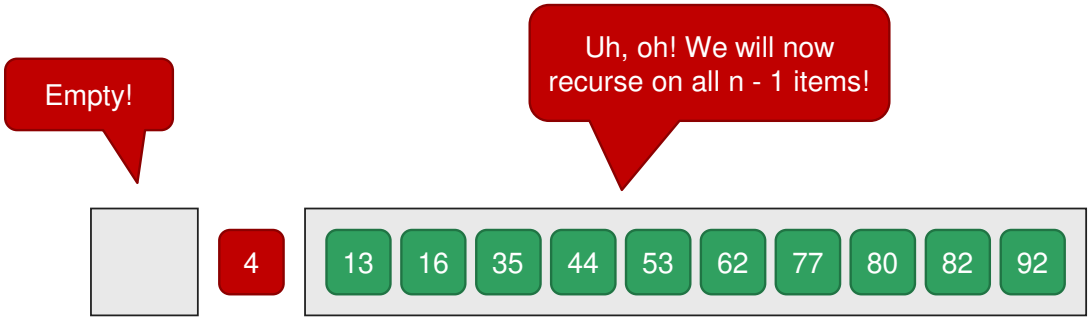
Worst Case: Now, Move Too Small



Worst Case: Pointers Passed



Worst Case: Recurse on n-1



Quick Sort Analysis

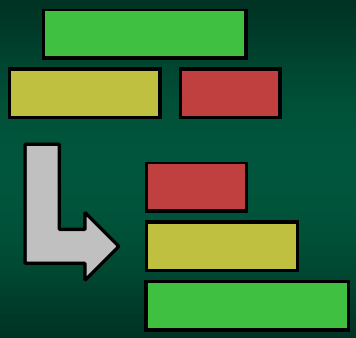
- So, in the worst case, Quick Sort is **$O(n^2)$**
- ... and, given all the work it has to do with the pointers, it gets beat by Bubble Sort

How Can We Avoid This?

- If you don't know if the array is randomized, *manually randomize the values*
- O(n) – run i from first to last element and swap array[i] and array [random]

Quick Sort Summary

Quick Sort	
Time Average	O(n log n)
Time Best	O(n log n)
Time Worst	**$O(n^2)$**
Auxiliary space	O(1)
Stable	No – Equal element order not preserved
Online?	No

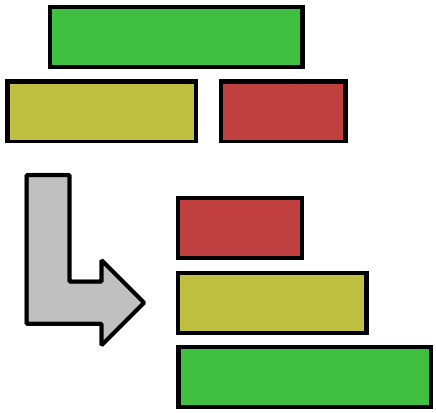


Heap Sort

Heap-a-rific algorithm!

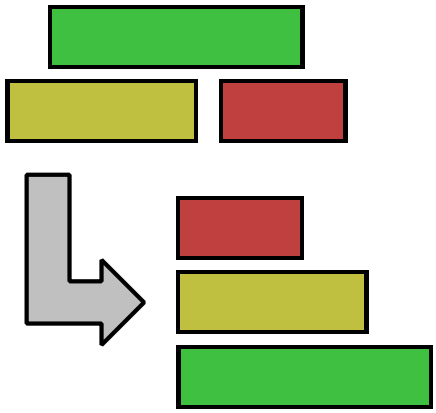
Heap Sort

- *Heap Sort* is an ingenious algorithm that uses a max-heap to sort an array
- *John W. J. Williams* invented both heaps and the Heap Sort in 1964 *(0 ABW)*
- The same year, the sort was improved by *Robert Floyd*



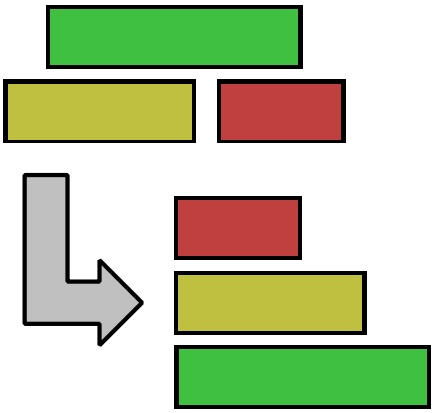
Heap Sort

- Heap Sort takes advantage of the fact that a heap is a natural priority queue
- … and that a heap will always add / remove from the right-most leaf



Heap Sort Works in Two Phases

- Phase 1: Heapify
  - converts the existing array into a max-heap
- Phase 2: Empty the heap
  - removes all the nodes (treating it as priority queue)
  - sorted data is added to the end of the array



Implementation

- Both the "heap" and the remaining array can be used in memory at the same time
- The sorted array is stored at the empty space after the end of the heap
- This concept works for both Phase 1 and Phase 2

Phase 1: Array → Heap

- In Phase 1, we convert the array into a max-heap. This step is called *heapify*.
- Remember....
  - a heap can be stored in an array
  - so, we can just *look at the array as a heap*
  - ...but, its not quite a heap yet
  - data needs to be rearranged to turn the array into a heap

How do we convert it?

- First approach: *top-down*
  - start building the heap at the top of the array
  - iterate *i* starting at 0 and build a heap above *i*
  - item are upheaped
- Second approach: *bottom-up*
  - fastest approach is to downheap all the leaves
  - run the downheap, at the root, all the leaves

Phase 1: Heapify (top down)

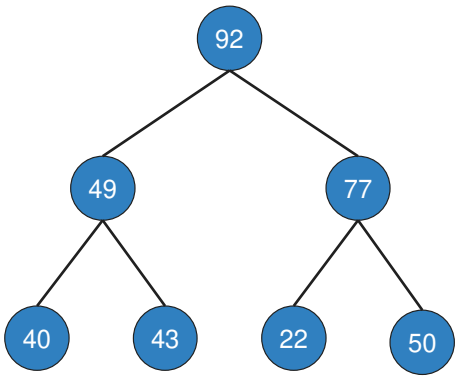
```
procedure heapify(array, count)
   last = count - 1
   n = 0                    //First item

   while (n <= last)
      upHeap(array, 0, n - 1)
      n++
   end while
end procedure
```


Phase 1: Heapify (top down)

```
procedure heapify(array, count)
   last = count - 1
   n = last                    //last item

   while (n >= 0)
      downHeap(array, n + 1, last)
      n--
   end while
end procedure
```

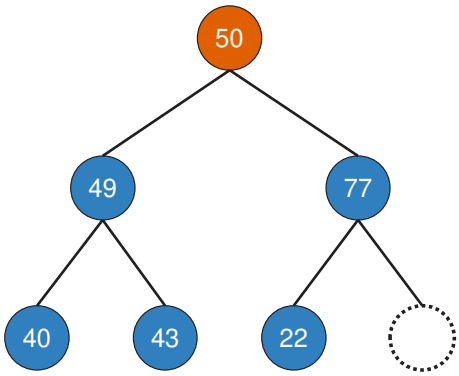
Phase 2: Root Deletion

- Now that the array is a **max-heap**, the root contains the maximum item
- If we remove the root, we have the *last item of the sorted array!*



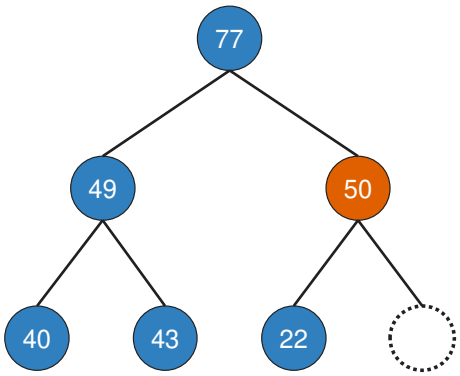
Phase 2: Root Deletion

- When we remove the root... right-most leaf is moved to the root
- …and then downheaped into the correct position



Phase 2: Root Deletion

- Now the root contains the second-largest item in the array
- *This leaf position is now empty*



Phase 2: Root Deletion

- We can put the root, that was just removed, in this new empty space
- *What a sec!* We just put the largest item in the last position in the array!

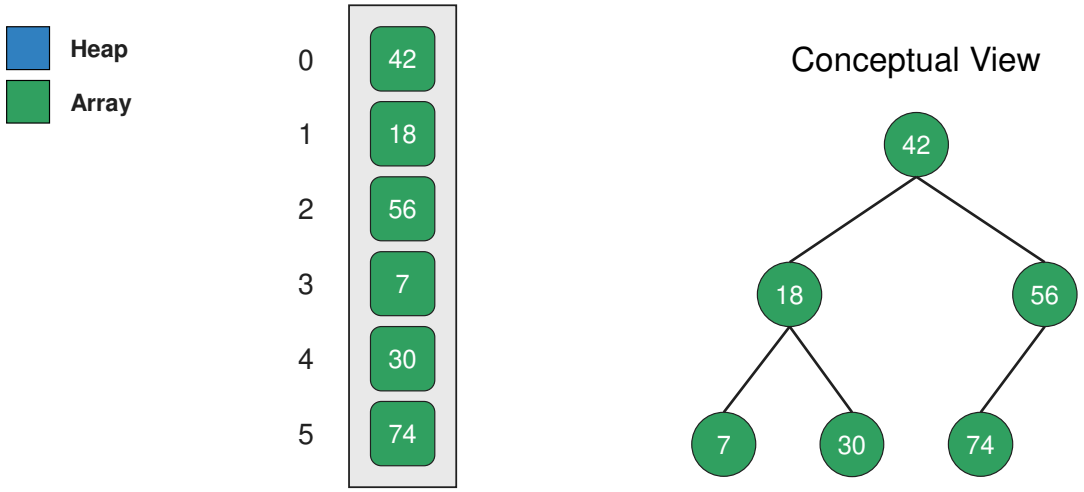
Phase 2: Root Deletion

- So, to sort the array....
  - so, we just keep removing the root and placing it position where the leaf was located
  - the "heap" section of the array shrinks as the sorted array grows from the bottom
- *OMG! Sooooo, awesome!*

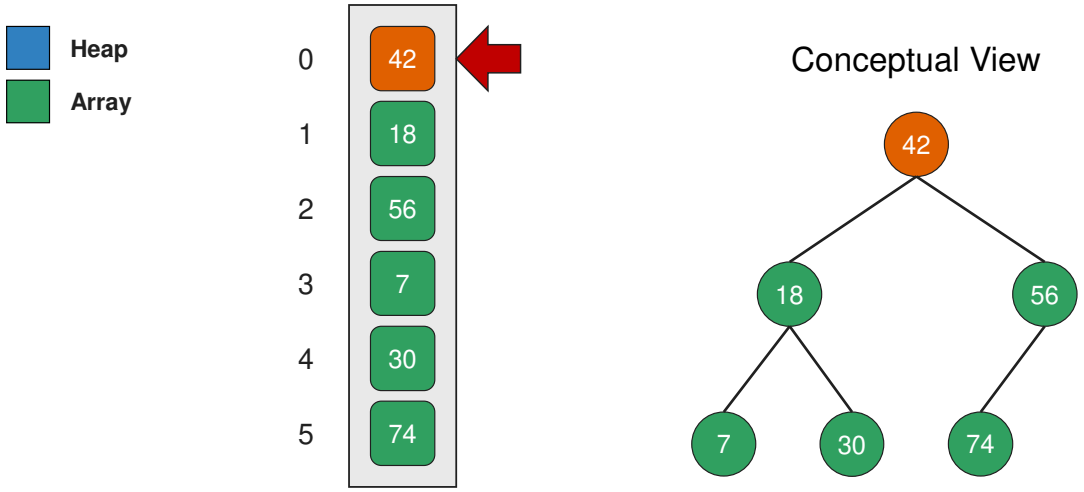
Heap Sort Algorithm

```
last = count - 1
heapify(array, 0, last)

while (last > 0)
   swap array[0] and array[last]
   downHeap(0, last - 1)
   last--
end while
```

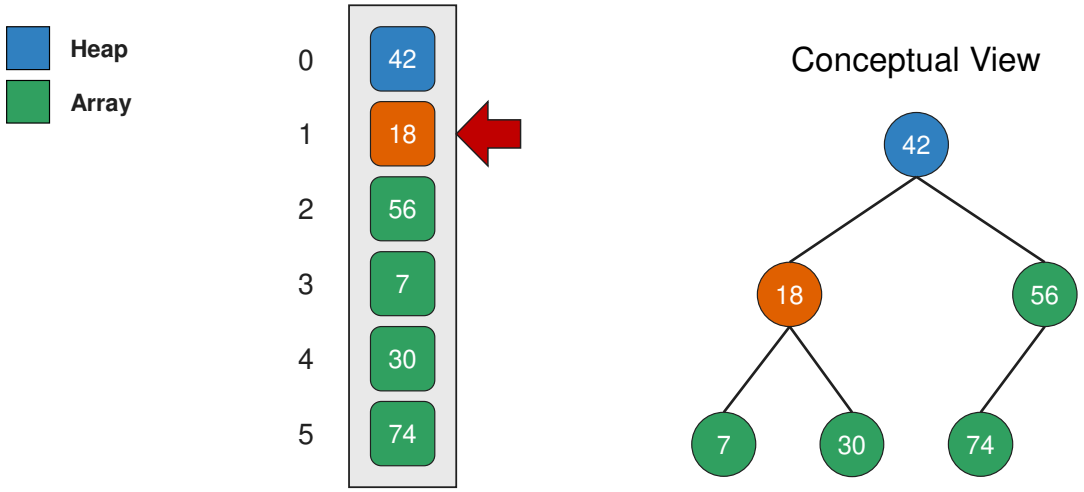
Phase 1 – Top-down Upheap



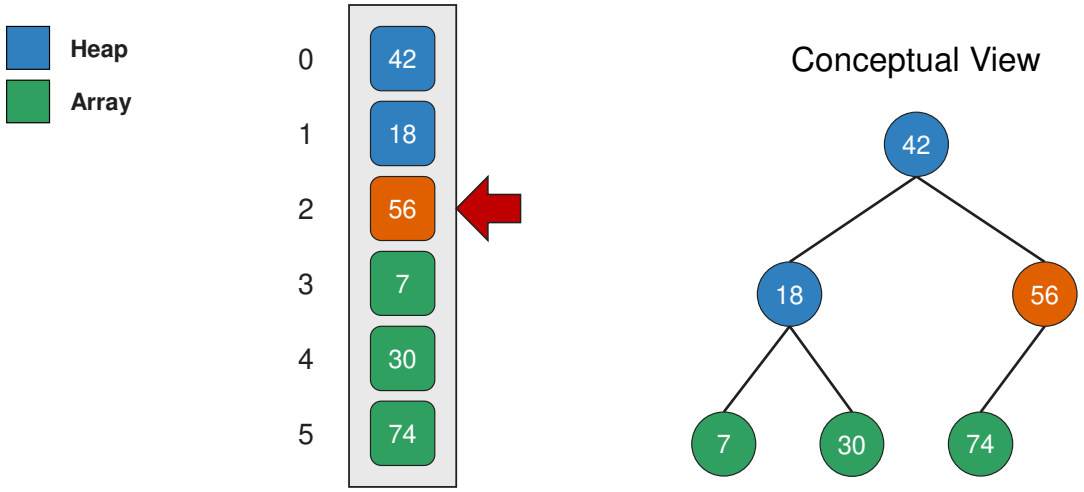
Phase 1 – Top-down Upheap



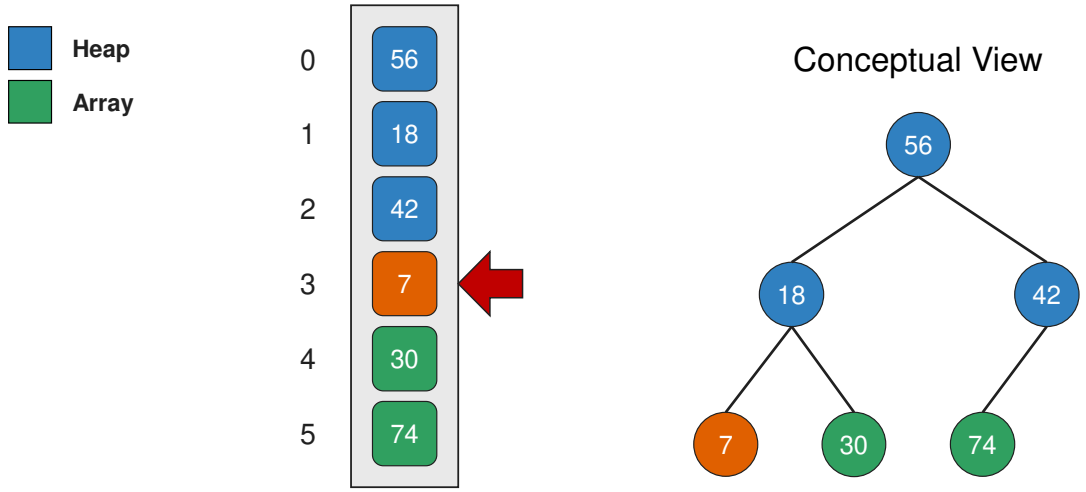
Phase 1 – Top-down Upheap



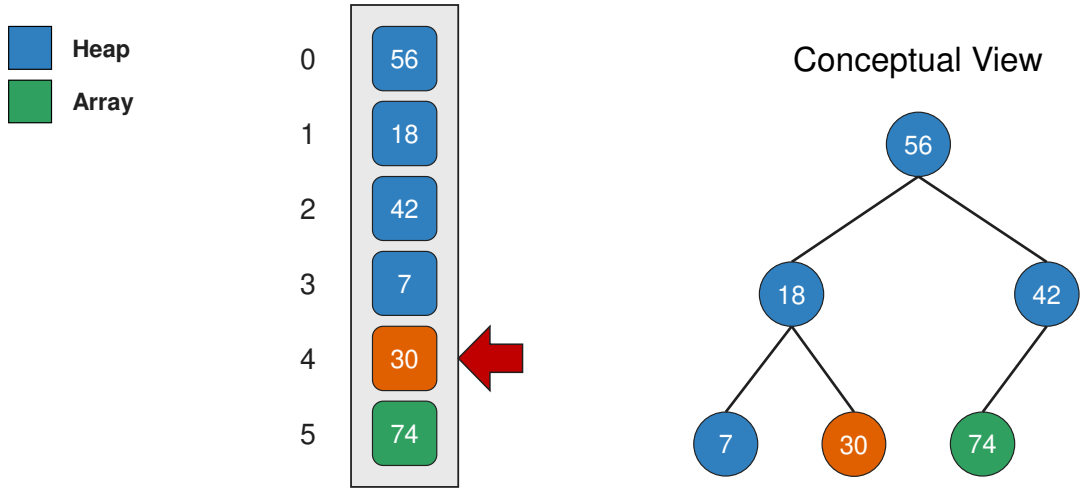
Phase 1 – Top-down Upheap



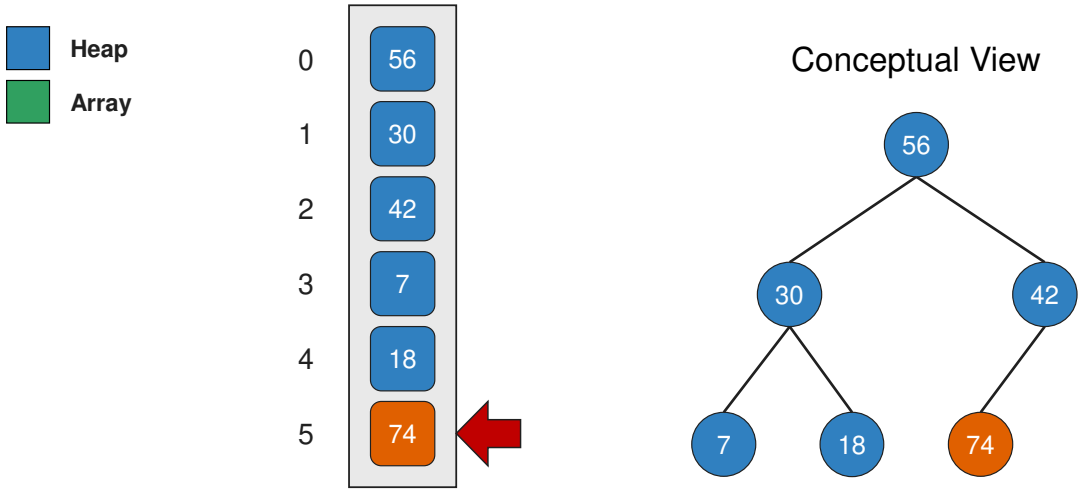
Phase 1 – Top-down Upheap



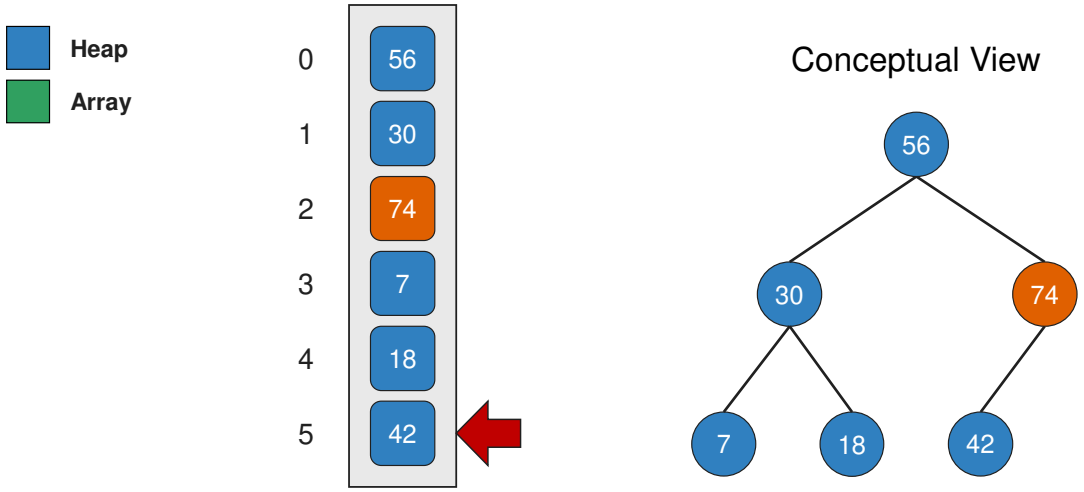
Phase 1 – Top-down Upheap



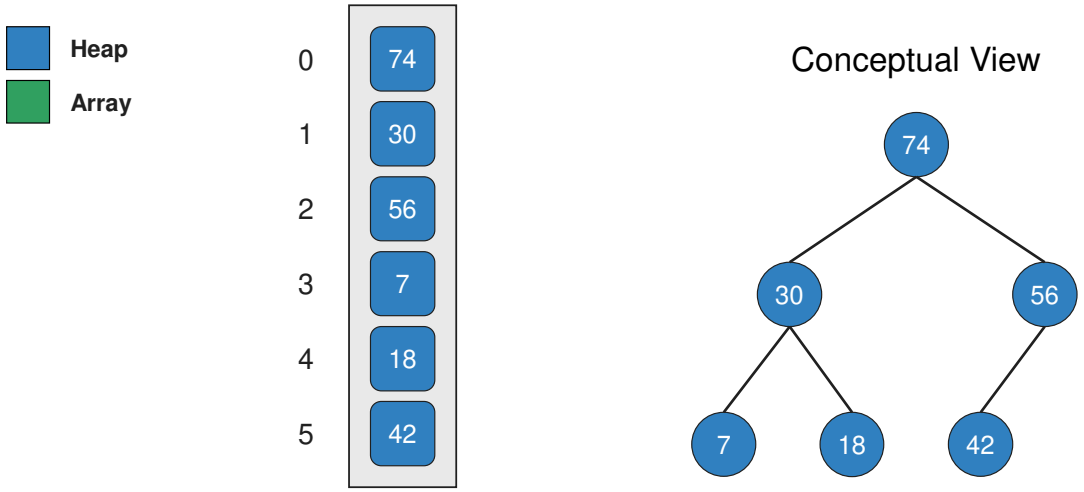
Phase 1 – Top-down Upheap



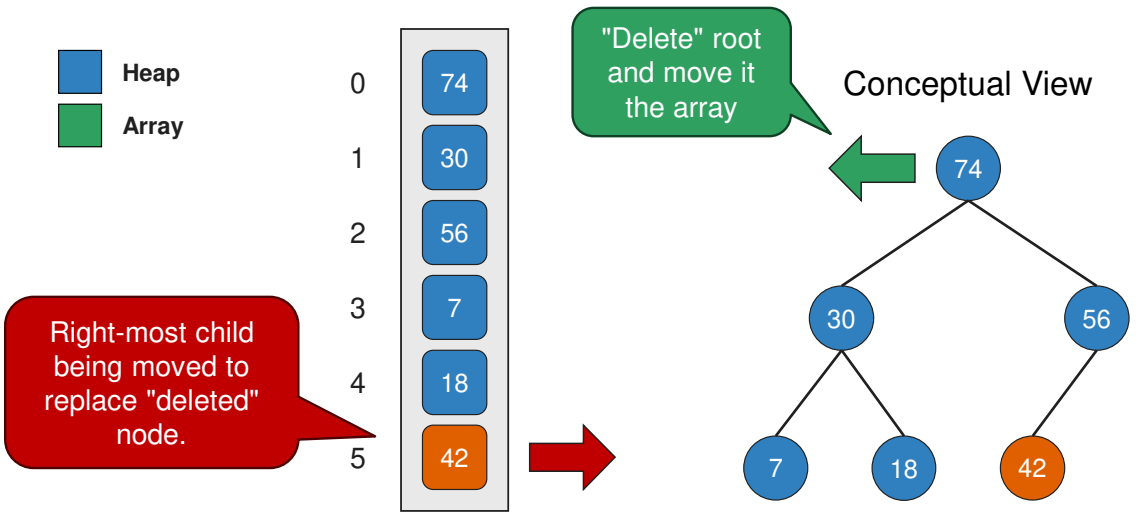
Phase 1 – Top-down Upheap



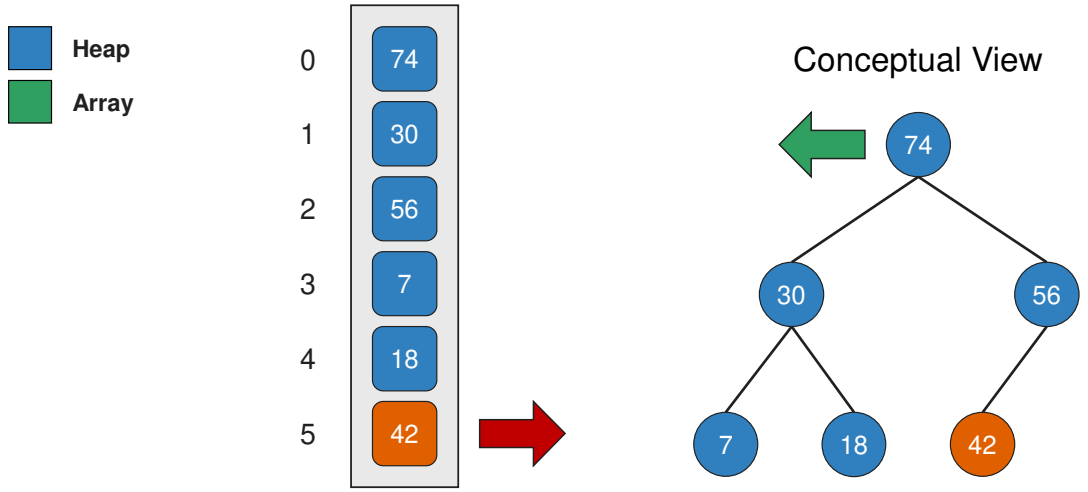
Phase 1 – Complete



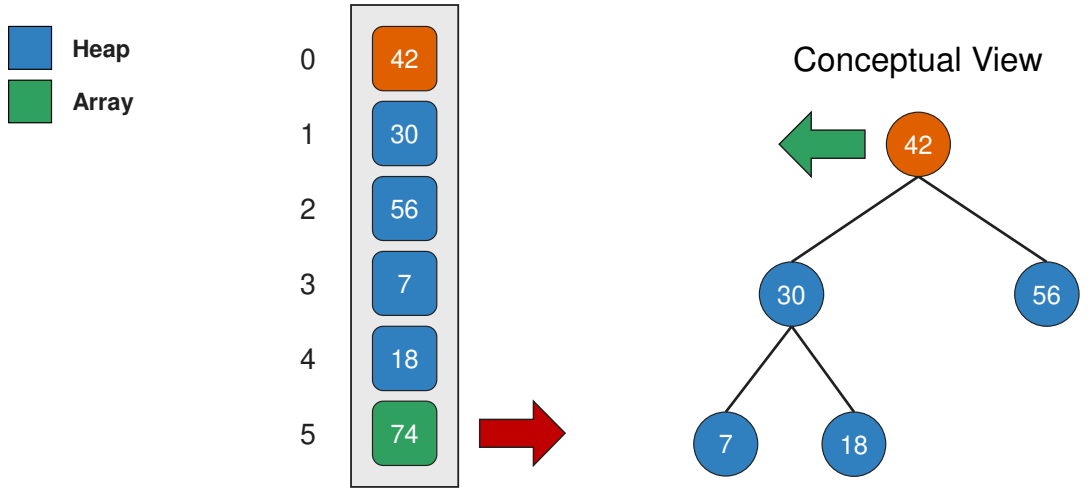
Phase 2 – Remove Root



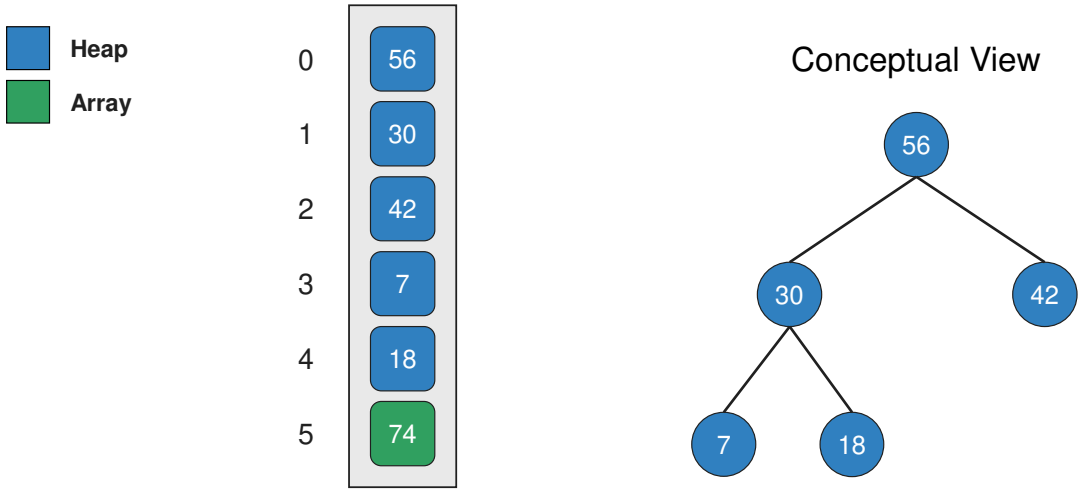
Phase 2 – Remove Root



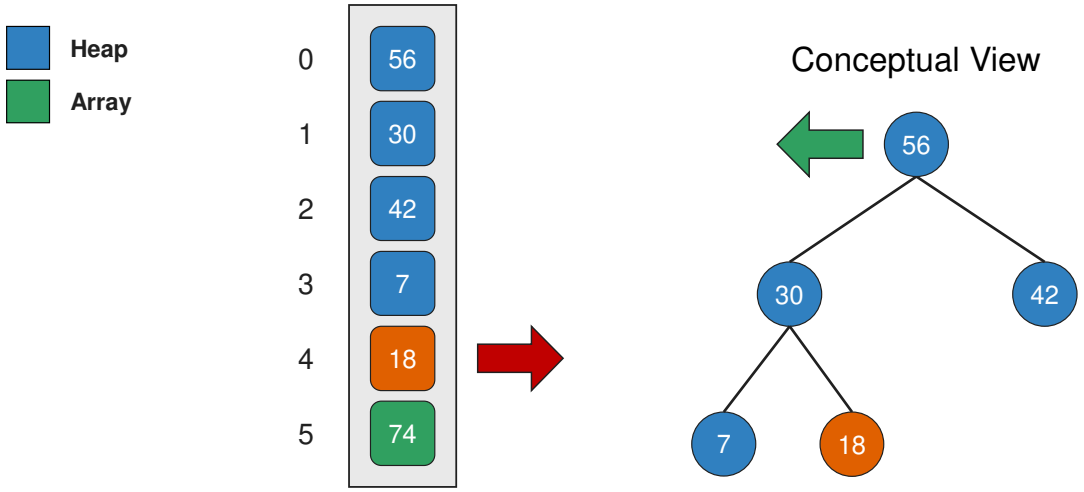
Phase 2 – Downheap



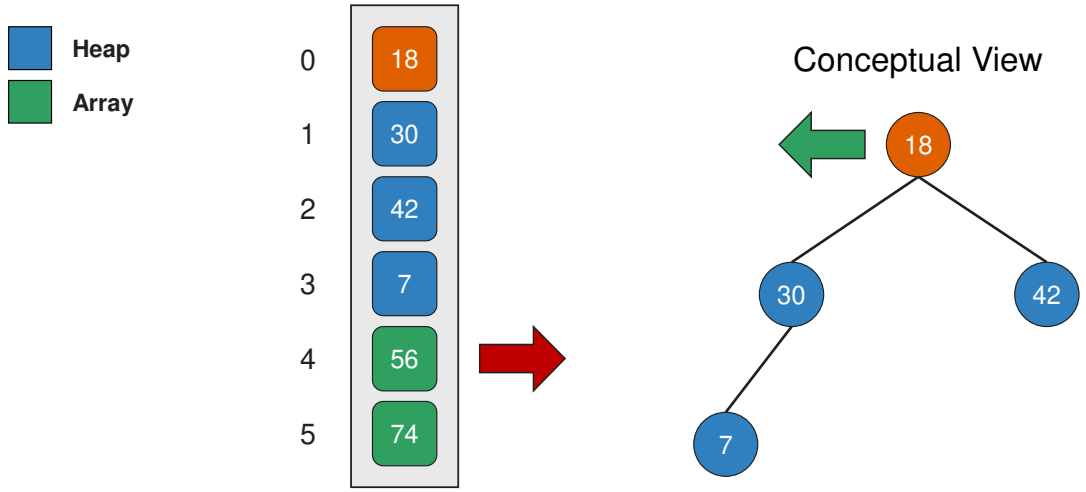
Phase 2 – Item Finished ✓



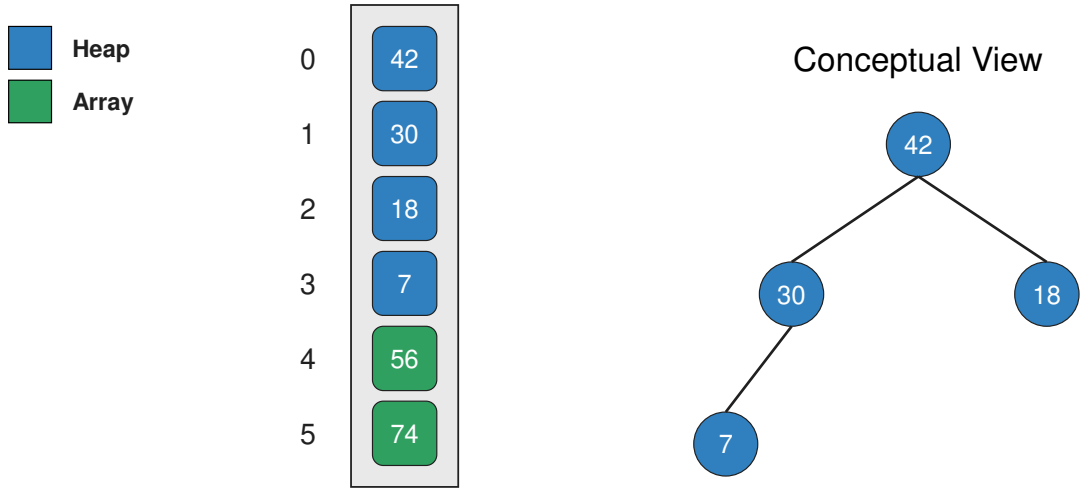
Phase 2 – Remove Root



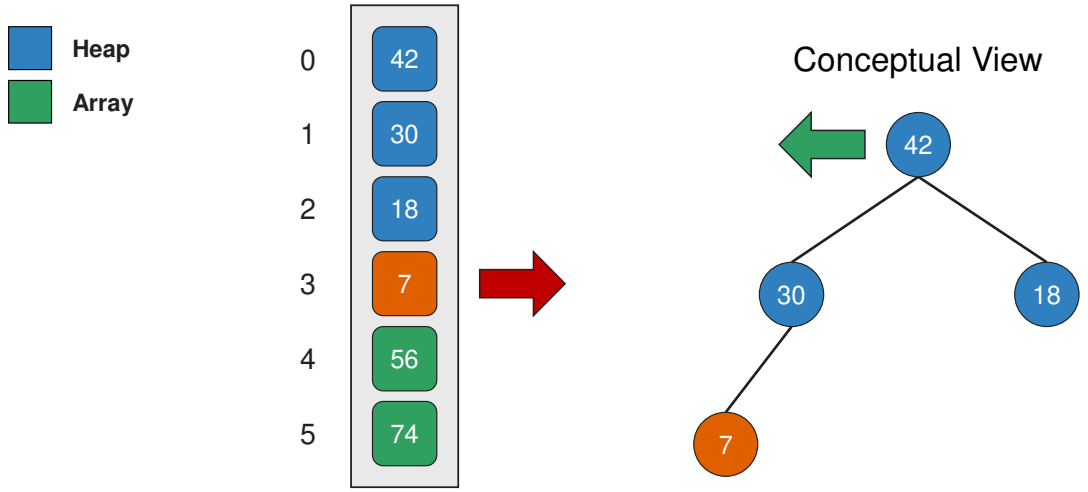
Phase 2 – Downheap



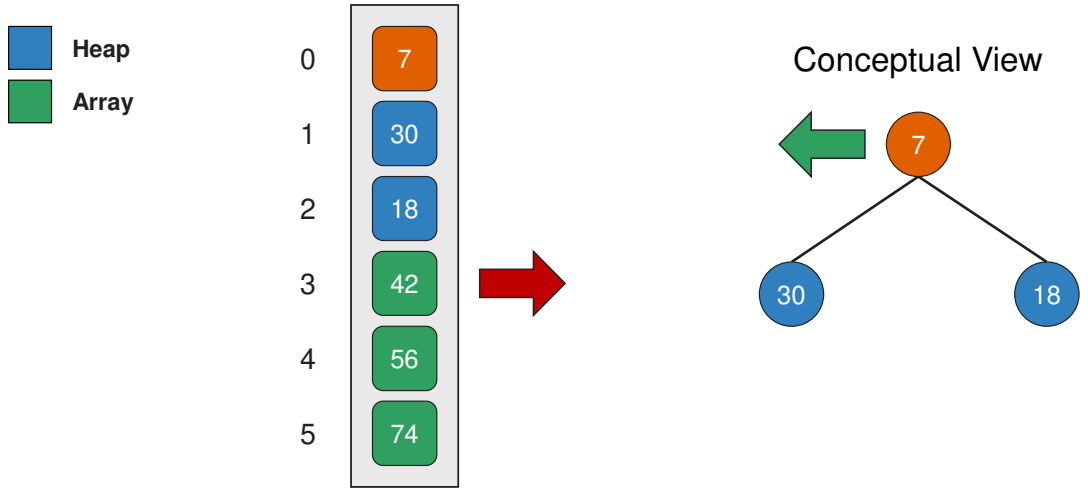
Phase 2 – Item Finished ✓



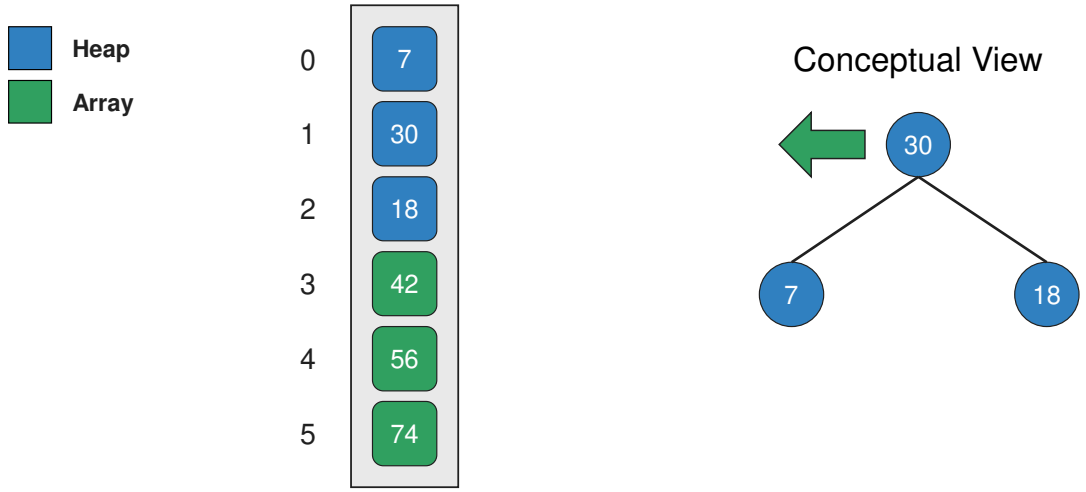
Phase 2 – Remove Root



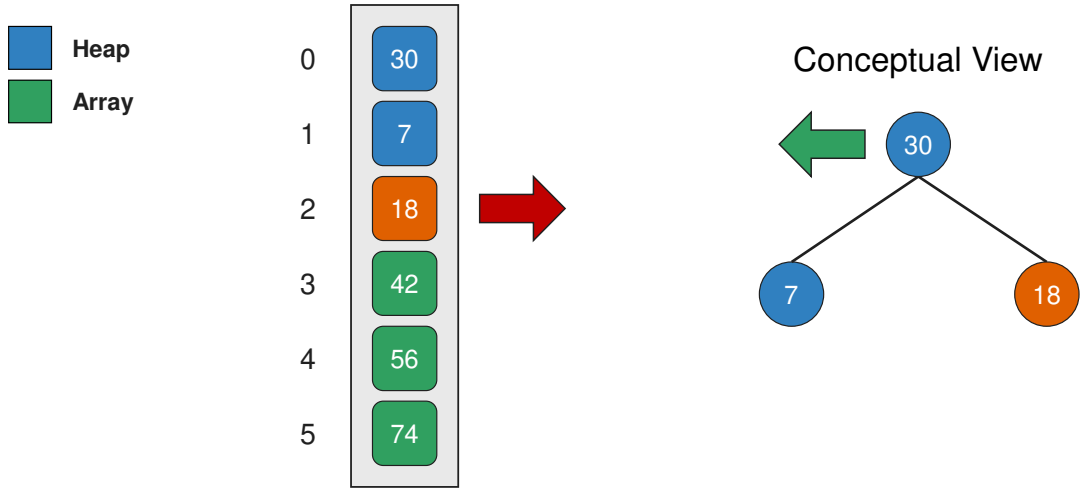
Phase 2 – Downheap



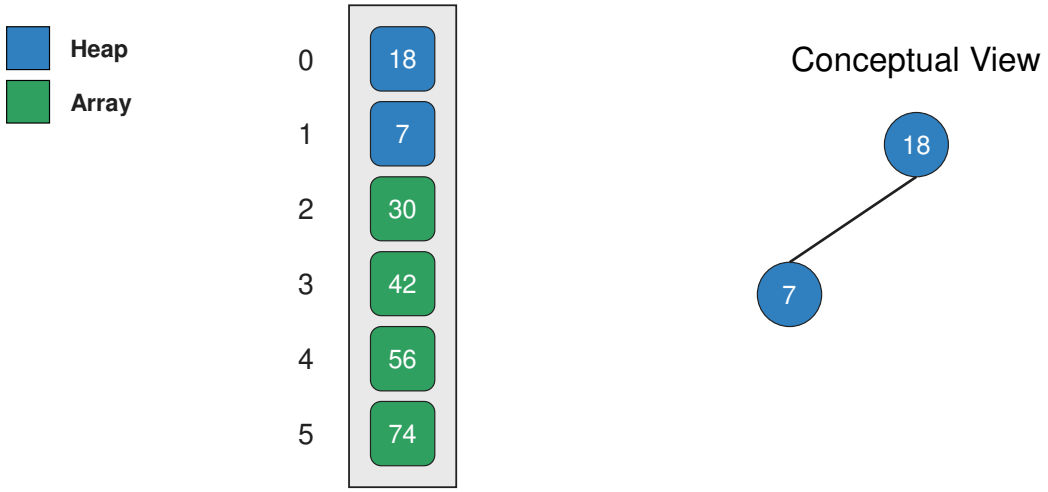
Phase 2 – Item Finished ✓



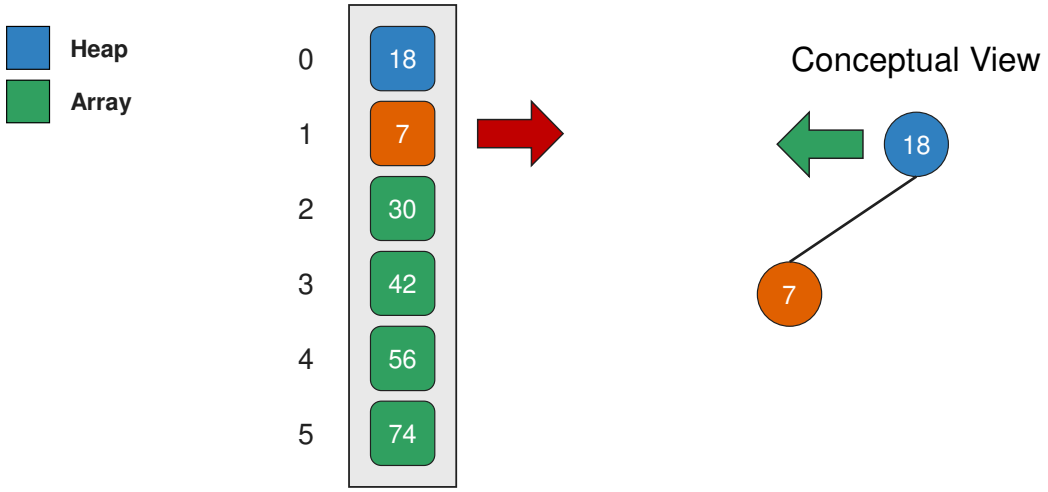
Phase 2 – Remove Root



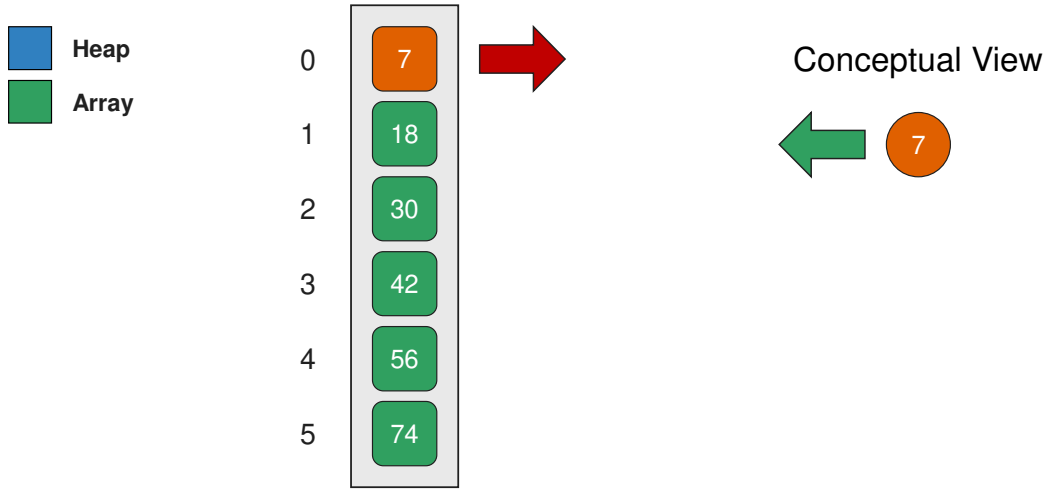
Phase 2 – Item Finished ✓



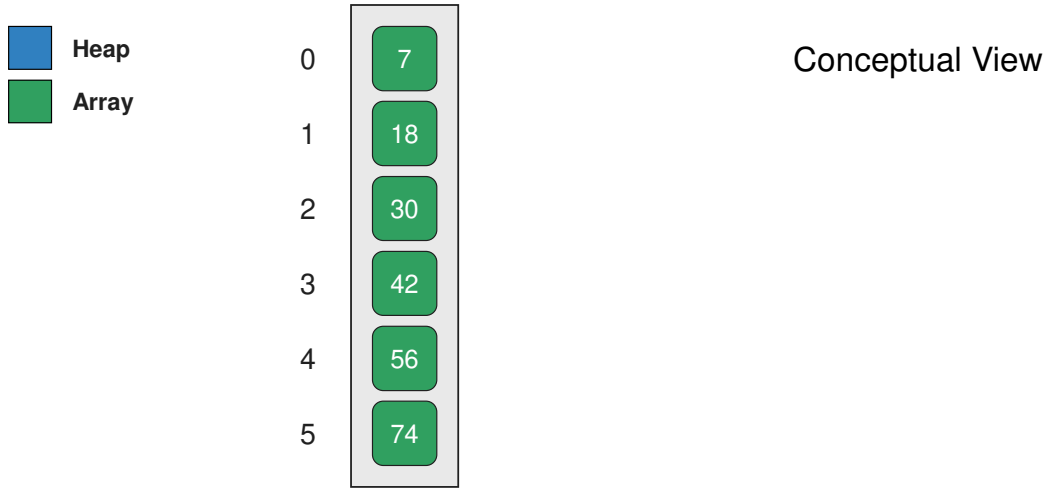
Phase 2 – Remove Root



Phase 2 – Remove Root



Complete



Merge Sort vs. Heap Sort

- Heap-Sort allows us to sort any array in O(n log n) just like Merge-Sort & Quicksort
- However, there is no overhead
  - Heap-Sort can be sorted in-place, meaning auxiliary storage is O(1)
  - Merge-Sort, however, requires O(n)
  - Quick-Sort can become $O(n^2)$

Merge Sort vs. Heap Sort

- However, in some cases, the recursive nature of Merge Sort is better
  - easy to distribute to multiple computers
  - Heap-Sort uses the entire array – *not online*
- But...in the Real World, it gets complex
  - you can cut an array into sub-lists, send them to different machines which Heap-Sort them
  - ... and then you Merge

Heap Sort Summary

Heap Sort	
Time Average	O(n log n)
Time Best	O(n log n)
Time Worst	O(n log n)
Auxiliary space	O(1)
Stable	No – Equal element order not preserved
Online?	No